Graph Theory

Graph Theory & Trees

Defn: A Graph is written as $G = G(V, E)$ consist of two componants

(i) The finite set of vertices V (Also called as node, or point)

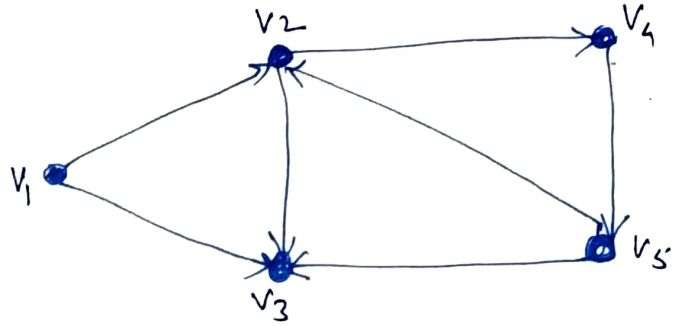
and (ii) The finite set of edges E (Also called as line, or arc connecting a pair of vertices).

Directed graph :⇒

A graph is said to be the directed graph or di-graph if it is required to associate a direction with edge of the graph.

Edges are ordered pairs of distinct vertices.

A directed graph can be regarded as one-way road where traffic flow is permited in only one direction.



Here. V_1, V_2, V_3, V_4, V_5 are vertices of graph and order pair (V_1, V_2), (V_1, V_3), (V_2, V_3), (V_2, V_4), (V_4, V_5), (V_5, V_3) are edges of the graph.

Un-directed graph :⇒

It is a graph in which edges betn two vertices are denoted by a un-directed line, and assumed that it is two way road where traffic flow is permitted in both direction

for eg : If $e = \{v_i, v_j\}$ ie edge from v_i to v_j
or edge from v_j to v_i

for eg :



Directed graph

undirected graph

Loops :

Indegree of vertex

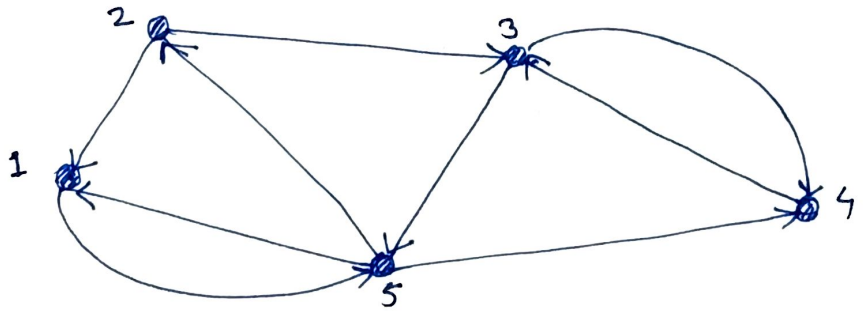
The indegree of a vertex V in a di-graph is the number of edges ending on it.
and it is denoted by indeg.(V)

Outdegree of vertex

The outdegree of vertex V in a di-graph is the number of edges begining from it.
and it is denoted by outdeg.(V).

Degree of Vertex: it is the total number of edges connecting with the vertex.

for:



Vertex	indeg.(V)	outdeg(V).	degree.
1	2	1	3
2	1	2	3
3	2	2	4
4	2	1	3
5	2	3	5

Loops :→

A loop is the edge in a graph whose end & Starting points are same i.e it is an edge from a vertex to it self.

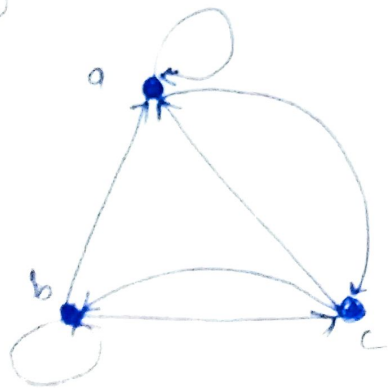


The indegree of loop is 1, and outdegree is 1

Hence A loop contributes 2 to the degree of vertex

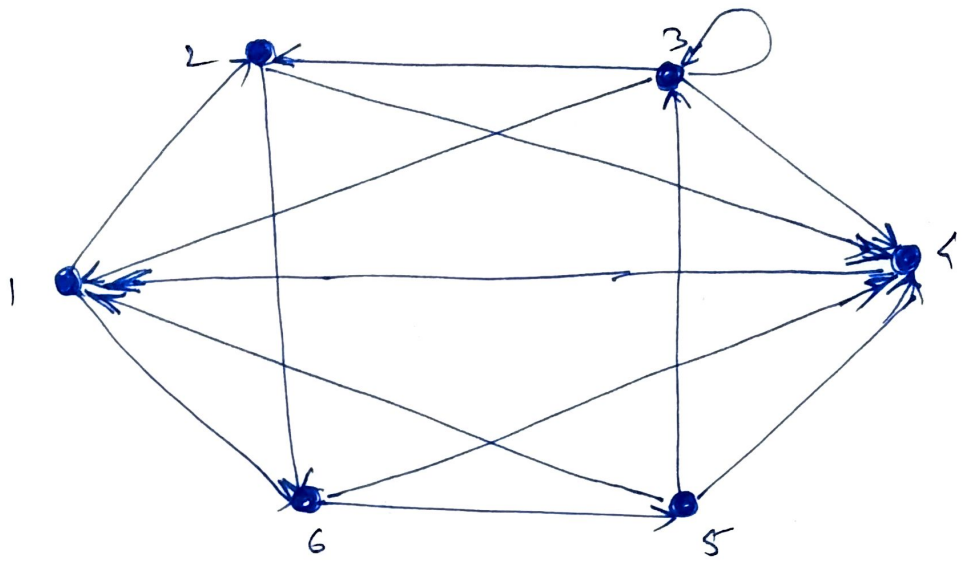
Ex: find indegree, out-degree & degree of vertices of following graphs

(1)



Vertex	indeg.	outdeg.	deg.
a	3	2	4+1
b	2	3	5
c	2	2	4

(ii)

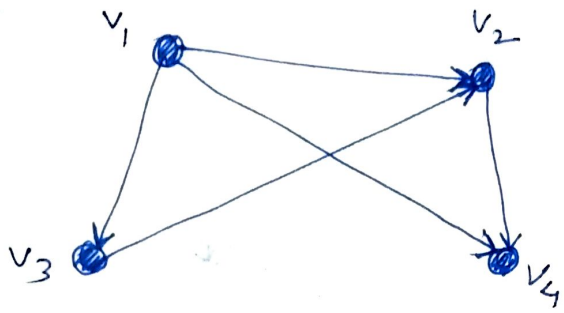


Vertex	Indegree	outdegree	Total degree
1	3	2	5
2	2	2	4
3	2	4	6
4	4	1	5
5	1	3	4
6	2	2	4

Simple graph :⇒

It is a di-graph in which two vertices are connected with only one ~~dege~~ edge and there is no loop.

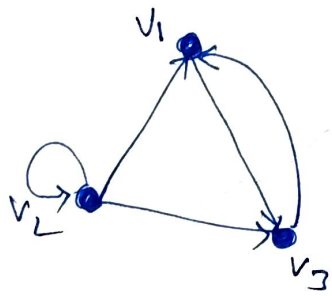
for eg.



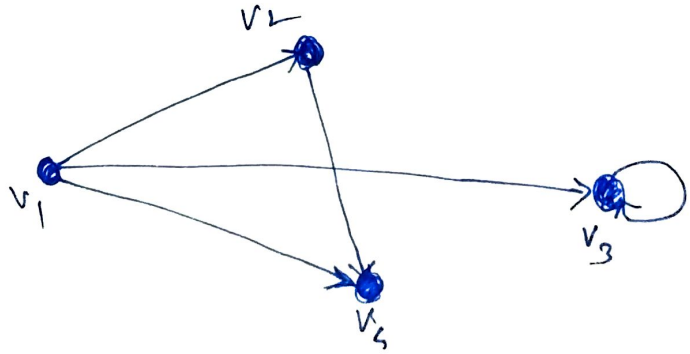
Multigraph :⇒

If there are multiple edges (two) connecting any two vertices or if there is a loop in a digraph, then it is called a multigraph.

For eg: (i)



(ii)



Walk :⇒

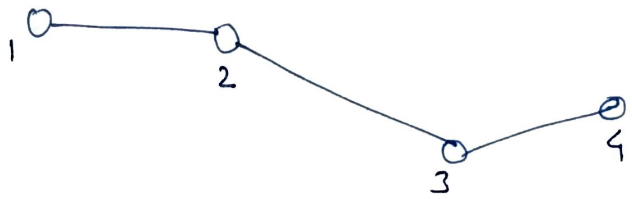
A walk in a graph $G(V, E)$ is a sequence $v_1, v_2, v_3 \ldots v_k$ of vertices each adjecent to the next. and a choice of an edge betn each v_i & v_{i+1} so that, no edge is chosen more than once.

OR

A walk is sequence of vertices and edges that ~~beigi~~ begins at v_1 and travel along edges to verten v_k. so that no edge appears more than once. However the verten may be appear more than ~~once~~. once.

for eg :

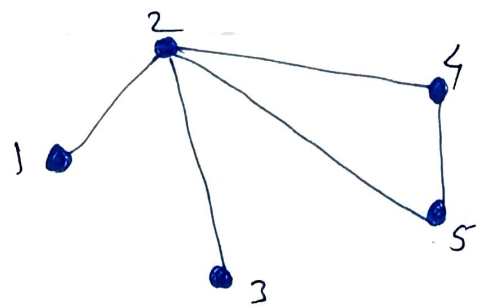
(i)



Then walk from vertex 1 to 4 is

1 — 2 — 3 — 4

(ii)



walk from vertex 1 to 3 are

1 - 2 - 4 - 5 . 2 - 3

& 1 — 2 - 3

Path:

A path is a walk through a sequence $v_1, v_2, \ldots v_k$ of vertices. each adjecent to the next without any repetition of vertices

For: from previous Example

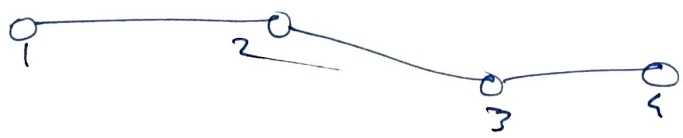
in graph ②

1 – 2 – 4 – 5 – 2 – 3

is not a path as vertex 2 repeated twice

but 1 – 2 – 3 . is a path. from vertex 1 to 3.

Path length: the number of edges in a path is called the length of the path

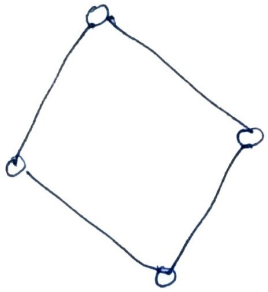


Path length {1, 4} is 3

Circuit: A closed walk is called a circuit which starts and ends at the same vertex. A circuit is also called circular path, or elementary cycle, or polygon or cycle.

for

(1)



(2)



$v_1 - v_2 - v_1$

(3)

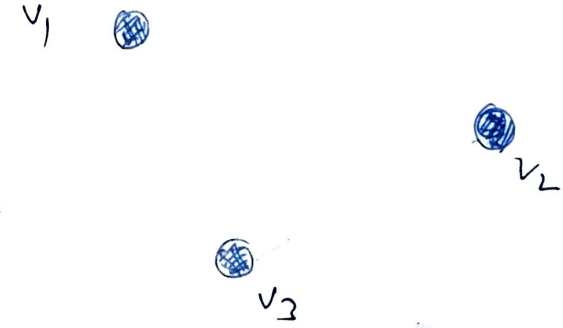


⇐ Types of Graphs :⇒

(1) Null graph.

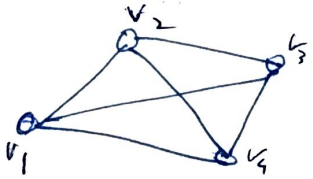
A graph is called a null graph if there is no edges or every vertex is an isolated vertex

for eg.



(2) Regular graph :

It is graph in which degree of each vertex are equal. If degree of each vertex is k then it is said to be k-Regular graph



$\Rightarrow$ 3-Regular graphs

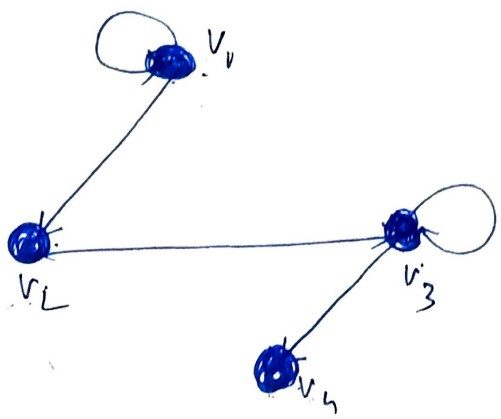
③ Weighted graph :⇒

A graph is said to be a weighted graph if a reqd number ω say (also called weight) is assigned to each of it's edge.

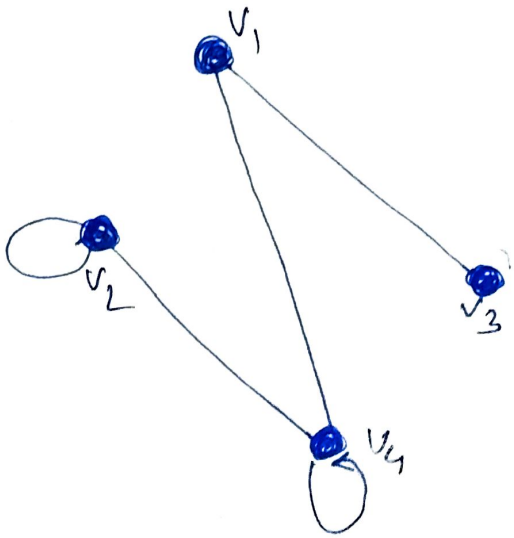
④ complementary graph :⇒

A complementary graph of graph G is denoted by $\overline{G}$ is the one which has same number of vertices. in which two of them are connected iff they are not connected in G.

Graph G



graph $\overline{G}$



(5) Complete graph :⇒

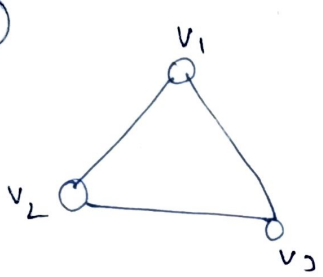
A graph in which an edge exists betn pair of different vertices is called a complete graph.

In otherwoord, In a complete graph, every vertex is connected to every other vertex.

A complet graph with n- number of vertices denote by K_n, and it has $\frac{n(n-1)}{2}$ number of edges.

For eg:

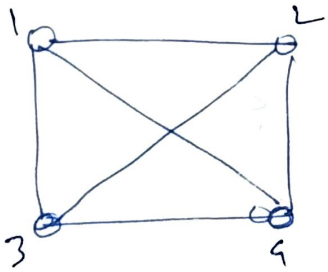
(i)



⇒ complete graph

$\frac{3(3-1)}{2} = 3$ edges

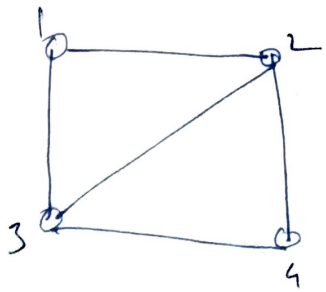
(ii)



⇒ Complete graph

$\frac{4(4-1)}{2} = \frac{12}{2} = 6$ edges

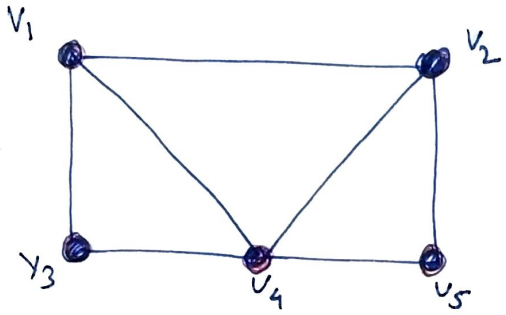
(iii)



⇒ not a complete graph because vertex 1, & 4 are not adjecent vertex (that is they are not connected by any edge)

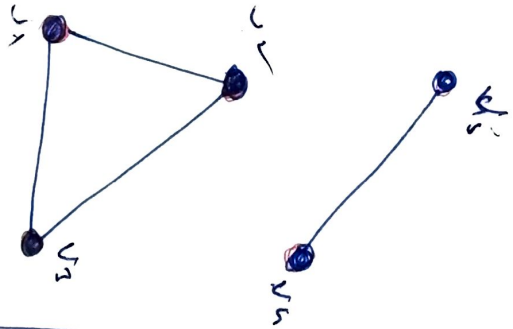
Connected graph :⇒

A graph G is said to be connected graph if there exist at least one path betn every pair of vertices. otherwise it is disconnected.



⇒ connected graph

but a graph

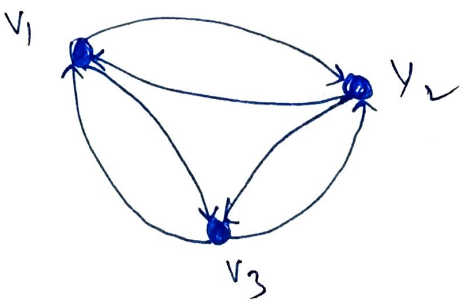


⇒ disconnected graph

Strongly connected graph :⇒

A directed graph is said to be strongly connected if for any pair of vertices V_i and V_j say there is a path from V_i to V_j and V_j to V_i

for eg:

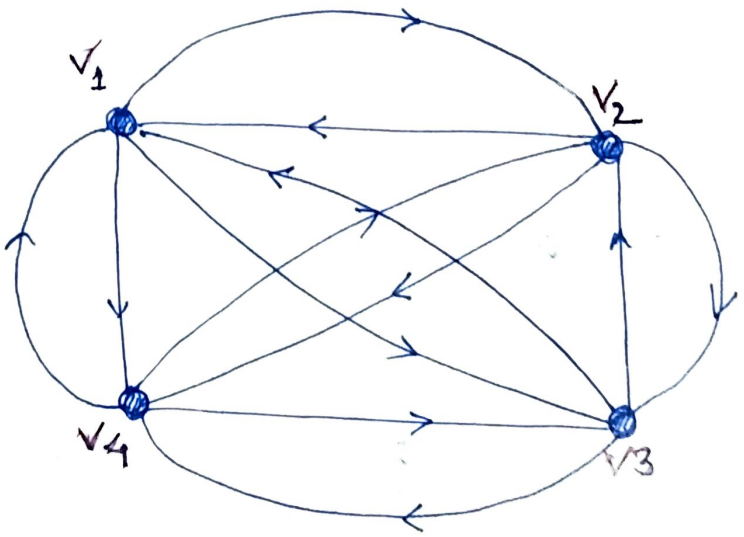


★ Reachability of Vertex

A vertex V_j in a di-graph is said to be reachable from the node V_i in graph G. if there exist a path from V_i to V_j.

The set of vertices which are reachable from a given vertex V_i is called the rechable set of V_i and it is denoted by $R(V_i)$

for ex :



$R(V_1) = \{V_2, V_3, V_4\}$

$R(V_2) = \{V_1, V_3, V_4\}$

Matrix Representation of a directed graph

(1) Adjacency matrix

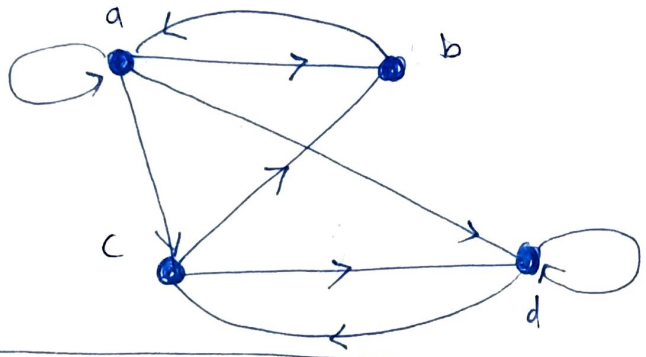
Let $G(V, E)$ be the directed graph having n vertices then the corresponding $n \times n$ matrix $A = \{a_{ij}\}$ whose elements given by

$$a_{ij} = \begin{cases} 1 & \text{if there is an edge from } i^{th} \text{ to } j^{th} \text{ vertex} \\ 0 & \text{, there is no edge from } i^{th} \text{ to } j^{th} \text{ vertex.} \end{cases}$$

Ex:

(I)

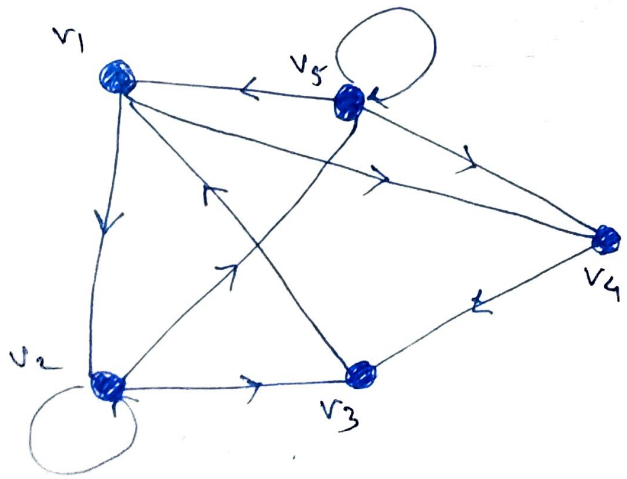
Graph



Adjacency matrix

$$A = \begin{array}{c} \\ a \\ b \\ c \\ d \end{array} \begin{array}{c} \begin{array}{cccc} a & b & c & d \end{array} \\ \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 \end{bmatrix} \end{array}$$

(II)



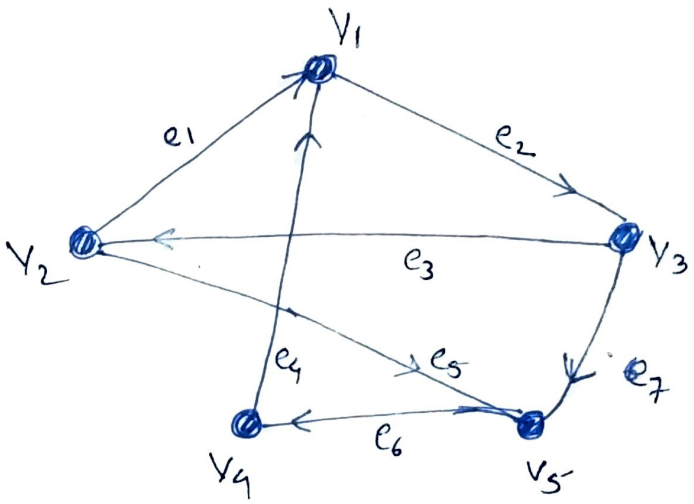
$$A = \begin{array}{c} \\ v_1 \\ v_2 \\ v_3 \\ v_4 \\ v_5 \end{array} \begin{array}{c} \begin{array}{ccccc} v_1 & v_2 & v_3 & v_4 & v_5 \end{array} \\ \begin{bmatrix} 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 \end{bmatrix} \end{array}$$

Incidence matrix : (for Directed graph)

Let $G(V, E)$ be the directed graph having n vertices and m- edges then nxm matrix $B = [b_{ij}]$ is called incidence matrix whose element are given by

$$b_{ij} = \begin{cases} 1, & \text{if } v_i \text{ is initial vertex of } e_j \\ -1, & \text{if } v_i \text{ is final vertex of } e_j \\ 0, & \text{if } v_i \text{ not incident on } e_j \end{cases}$$

for eg.



Req'd incidence matrix

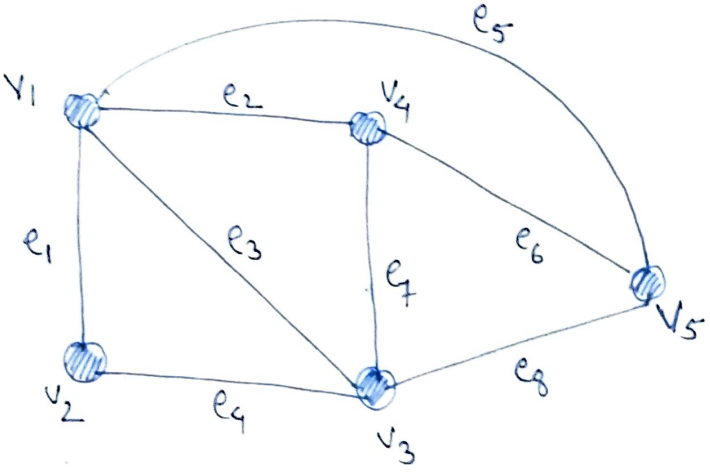
$$B = \begin{array}{c} \\ V_1 \\ V_2 \\ V_3 \\ V_4 \\ V_5 \end{array} \begin{array}{c} \begin{array}{ccccccc} e_1 & e_2 & e_3 & e_4 & e_5 & e_6 & e_7 \end{array} \\ \begin{bmatrix} -1 & 1 & 0 & -1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & -1 & -1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & -1 & 0 \\ 0 & 0 & 0 & 0 & -1 & 1 & -1 \end{bmatrix} \end{array}$$

Incedence matrix (for un-directed graph)

let $G(V, E)$ be the un-directed graph having n vertices and m edges then $n \times m$ matrix $B = [b_{ij}]$ is called incedence matrix whose element are given by

$$b_{ij} = \begin{cases} 1 & , \text{if } v_i \text{ is incedence on edge } e_j \\ 0 & , \text{otherwise} \end{cases}$$

for eg :⇒



Incedence matrix

$$B = \begin{array}{c} \\ v_1 \\ v_2 \\ v_3 \\ v_4 \\ v_5 \end{array} \begin{array}{c} \begin{array}{cccccccc} e_1 & e_2 & e_3 & e_4 & e_5 & e_6 & e_7 & e_8 \end{array} \\ \begin{bmatrix} 1 & 1 & 1 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 & 1 & 1 & 0 \end{bmatrix} \end{array}$$

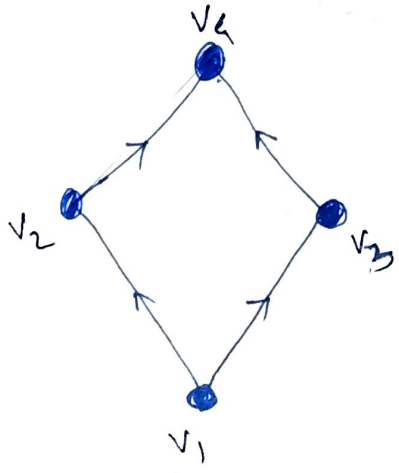
Isomorphic Graph

let G_1 and G_2 be the two distinct graphs. these two graphs are said to be isomorphic to each other, if there exist finite no. of vertices and edges and there is one-one correspondance bet^n the vertices as well as bet^n the edges of both. the graph, provided no. of vertices and no. of edges in both the graphs are same.

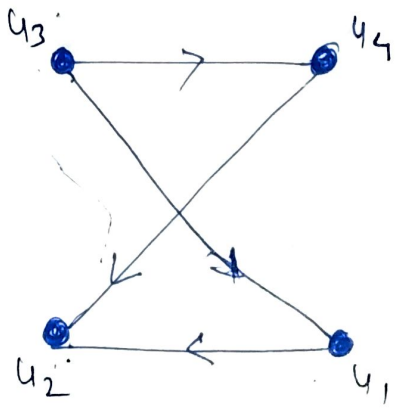
i.e. if $G_1(V, E)$ and $G_2(V', E')$ be the two graph then G_1 and G_2 are said to it isomorphic if there exist a mapping $f: V \longrightarrow V'$ and $g: E \longrightarrow E'$ are one-one mapping

for eg

Graph ①



Graph ②



$(V_2 V_4) \rightarrow (U_3 U_1)$



Since In graph (I)

Vertex	Indegree	outdegree
v_1	0	2
v_2	1	1
v_3	1	1
v_4	2	0

In graph (II)

Vertex	Indegree	outdegree
u_1	1	1
u_2	2	0
u_3	0	2
u_4	1	1

From this it is clear that

If $f: G_1 \longrightarrow G_2$

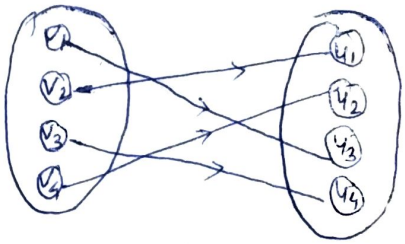
Then

$f(v_1) = u_3$

$f(v_2) = u_1\ u_4$

$f(v_3) = u_4\ u_1$

$f(v_4) = u_2$



i.e. all the vertex of G_1 one-one corresponding with vertex of graph G_2

Similarly In graph G_1 & G_2

$f(v_1\ v_2) = (u_3\ u_1)$

$f(v_1\ v_3) = (u_3\ u_4)$

$f(v_2\ v_4) = (u_1\ u_2)$ $\quad u_4$

$f(v_3\ v_4) = (u_4\ u_2)$ $\quad u_1\ u_2$

i.e all the edges of graph G_1 are one-one corresponds with edges of G_2

Thus graph G_1 and G_2 are isomorphic

i.e $\boxed{G_1 \cong G_2}$

H.P.

Ex: Draw the di-graph corresponding to the given adjecency matrices

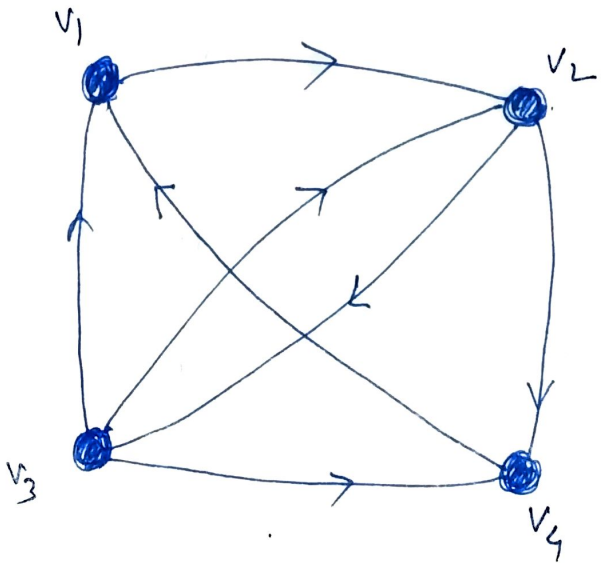
$$A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{bmatrix} \qquad B = \begin{bmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

Hence prove both graphs are isomorphic to each other.

Soln: Given adjecency matrices are

$$A = \begin{array}{c} \\ v_1 \\ v_2 \\ v_3 \\ v_4 \end{array} \begin{array}{c} \begin{array}{cccc} v_1 & v_2 & v_3 & v_4 \end{array} \\ \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{bmatrix} \end{array}$$

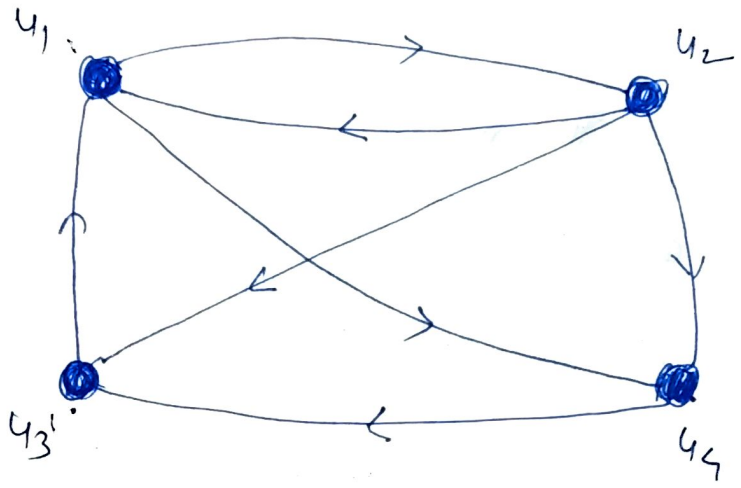
Thus digraph G_A is



also. for adjacency matrix

$$B = \begin{array}{c} \\ u_1 \\ u_2 \\ u_3 \\ u_4 \end{array} \begin{array}{c} \begin{array}{cccc} u_1 & u_2 & u_3 & u_4 \end{array} \\ \begin{bmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \end{array}$$

∴ digraph G_B



From graph G_A & G_B

For G_A

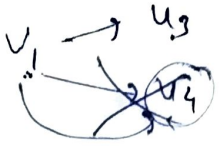
Vertex	Indegree	outdegree
V_1	2	1
V_2	2	2
V_3	1	3
V_4	2	1

For G_B

Vertex	Indegree	outdegree
u_1	2	2
u_2	1	3
u_3	2	1
u_4	2	1

Hence it is observed that

If $f: G_A \longrightarrow G_B$



Then

$f(v_1) = u_3$ ~~or u_4~~

$f(v_2) = u_1$

$f(v_3) = u_2$

$f(v_4) = u_4$ ~~or u_3~~

$(v_1 v_2) \rightarrow (u_4 u_1)$

$(v_1 v_2) \rightarrow (u_3 u_1)$

and

$f(v_1 v_2) = (u_3 u_1)$ but $f(v_1 v_2) \neq (u_4 u_1)$

$f(v_2 v_4) = (u_1 u_4)$

$f(v_2 v_3) = (u_1 u_2)$ ✗

$f(v_3 v_1) = (u_2 u_3)$

$f(v_3 v_2) = (u_2 u_1)$

$f(v_3 v_4) = (u_2, u_4)$

$f(v_4 v_1) = (u_4 u_3)$

Since ~~all the~~ There is one-one correspance betn the vertices of both the graphs as well as betn the edges of both the graphs

$\therefore$ G_A and G_B are isomorphic

$\therefore$ $\boxed{G_A \cong G_B}$

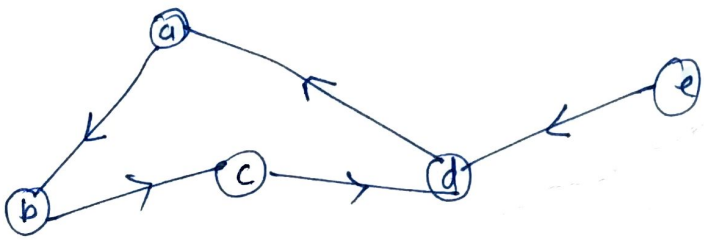
Examples for practice

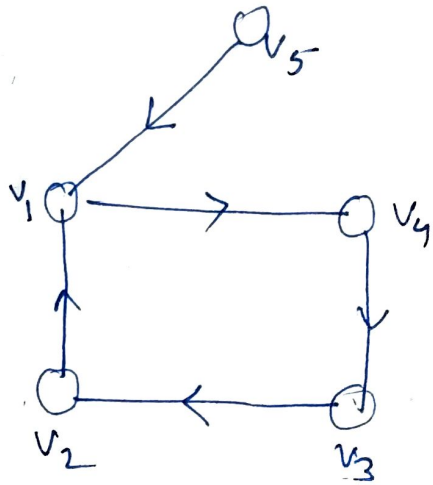
(1) Draw digraphs for the following matrices and Hence show both graphs are isomorphic.

(i) $$A = \begin{bmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{bmatrix} \qquad B = \begin{bmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}$$

(ii) $$M_1 = \begin{bmatrix} 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 & 0 \end{bmatrix} \qquad M_2 = \begin{bmatrix} 0 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 & 0 \end{bmatrix}$$

Ex: Show that following graphs are isomorphic





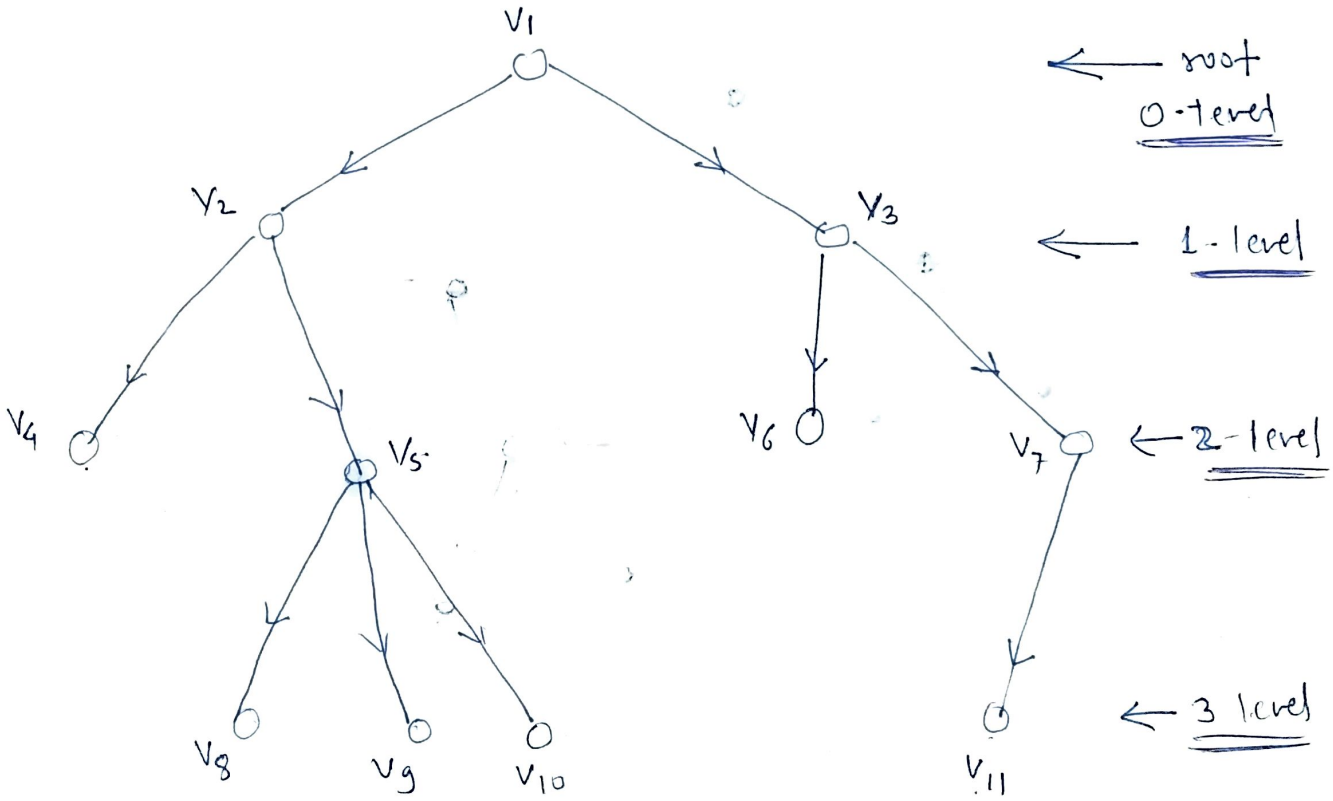
Trees

A tree is a connected acyclic graph.

A directed tree is an acyclic di-graph which has one node (called it's root) with indegree 0. while all other nodes (vertices) have indegree 1

In a directed tree any node which has outdegree 0 is called a leaf (i.e terminal node) and all other nodes are called branches nodes.

The level of any node is the length of it's path from the root. and the no. of levels of tree is called hight of tree.



Here: $\{V_4, V_6, V_8, V_9, V_{10}\}$ are leafs of tree
$\{V_2, V_3, V_7, V_5\}$ are branches & V_1 is root

★ Tree of an algebraic expression (structures)

Ex: Draw the tree for algebraic expression

(i) $(a+b) * [(c \div d) - e]$

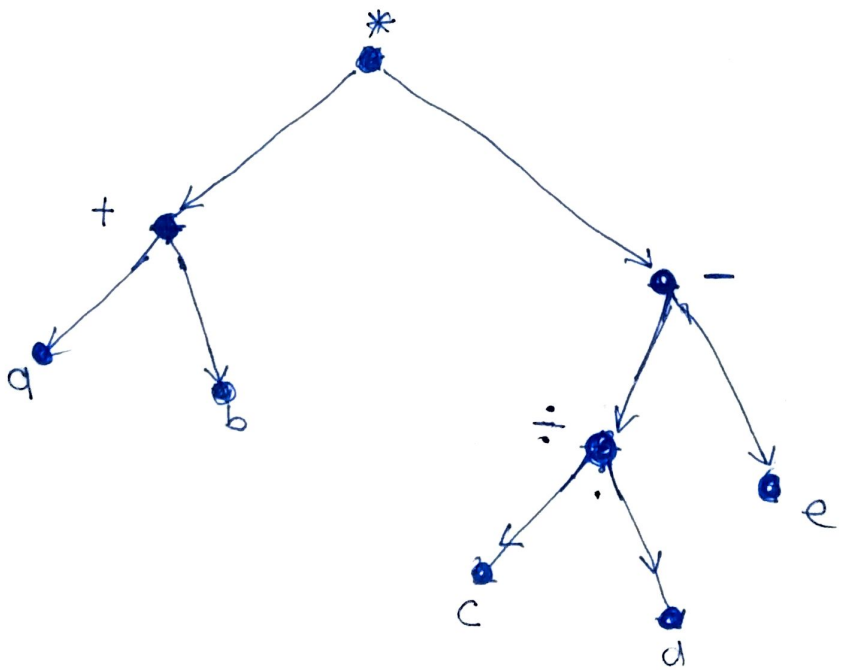
(ii) $((3 \times 4) + (2 \div 2)) - ((4 + 6(3 \div 2)) * (5 - 2))$

(iii) $(a+5) * [\{(7 * b) + c\} / (9 + d)]$

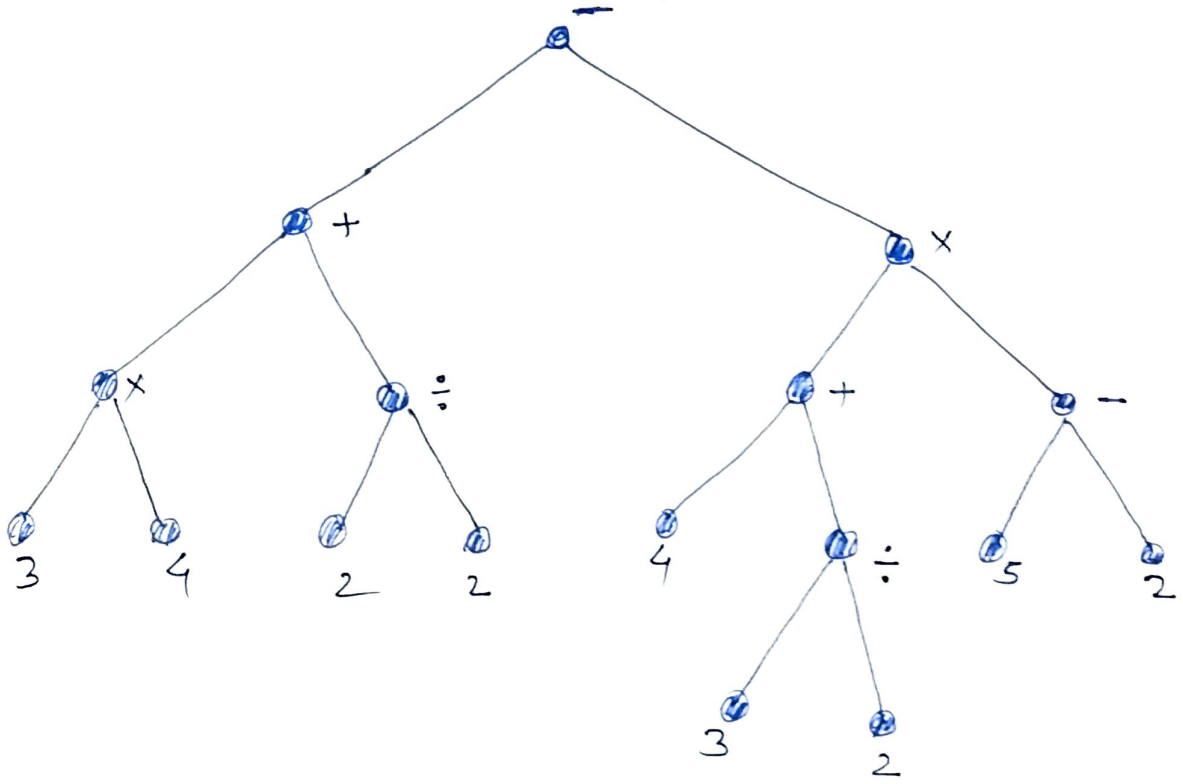
(iv) $(x - (y + z)) \div ((a + (b * c)) - d))$

Soln

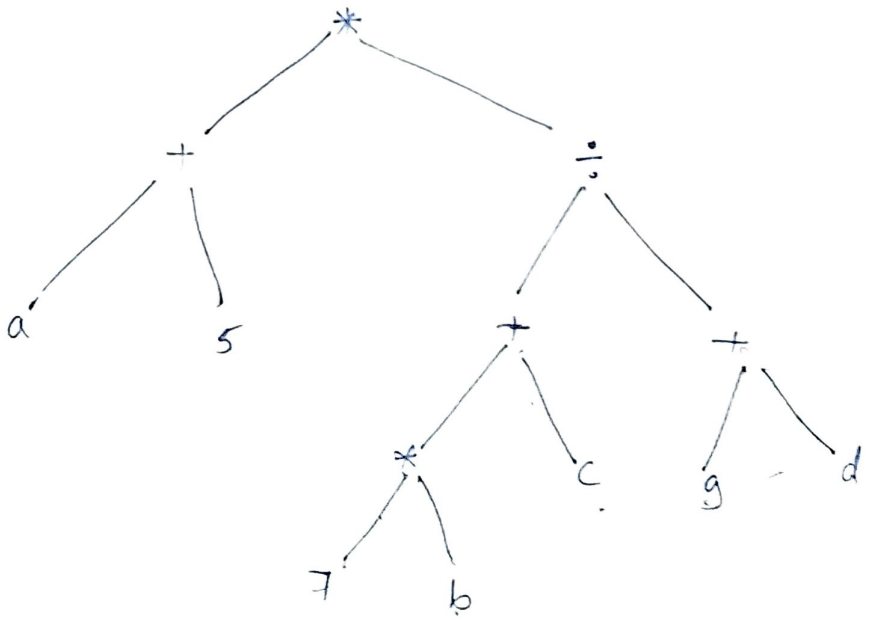
(i) $(a+b) * [(c \div d) - e]$



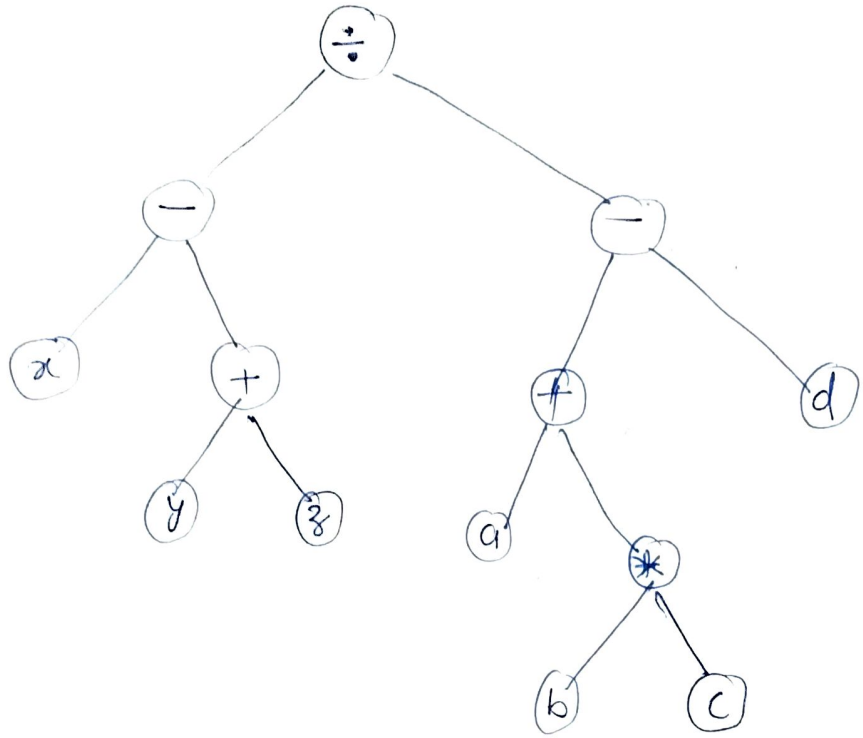
(ii) $[(3\times4)+(2\div2)]-[(4+(3\div2))\times(5-2)]$



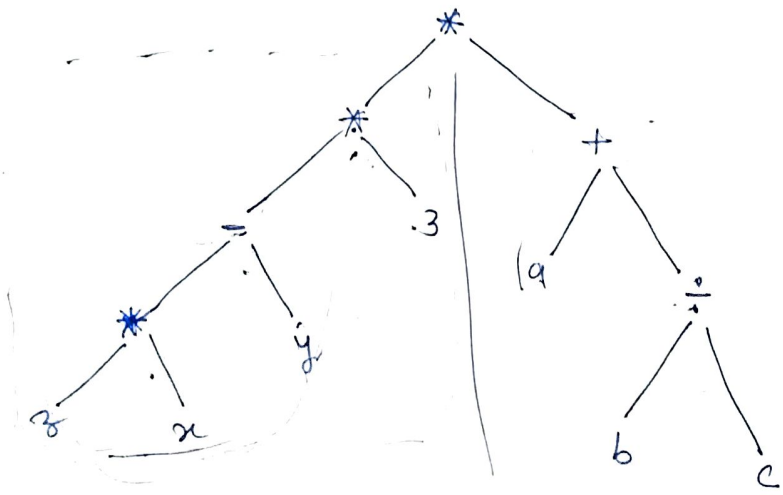
(iii) $(a+5) * [\{(7*b)+c\} / (g+d)]$



(iv) $[x - (y+z)] \div [(a + (b * c)) - d]$

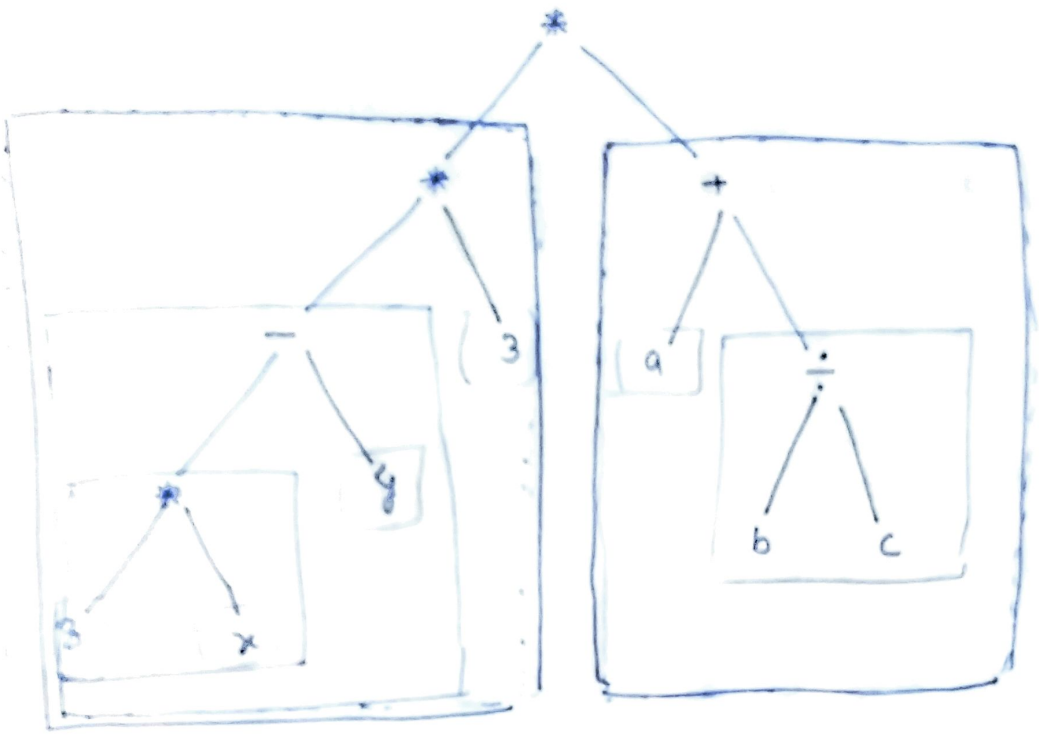


Ex: Obtain algebraic expression from following tree.



soln:

Given tree is



→

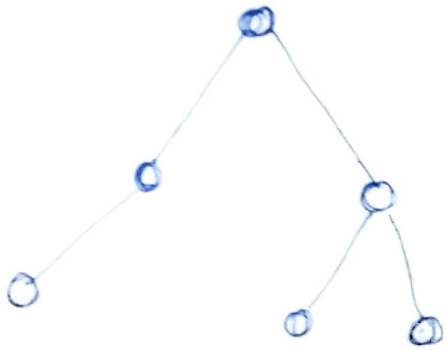
$$\Big(\big((3 * x) - y\big) * 3\Big) * \Big(a + (b \div c)\Big)$$



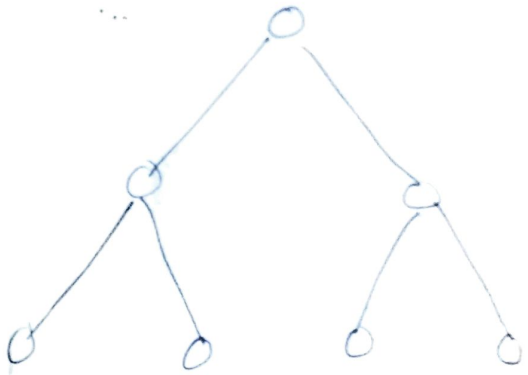
Binary Tree :⇒

If in a tree, outdegree of every node is less than or equal to 2 then the tree is called a Binary tree

for eg



(Binary tree)



(Complete Binary tree)

Working Method

step (I) Delete all the branches originating in the root node (initial / starting node) except the left most branch.

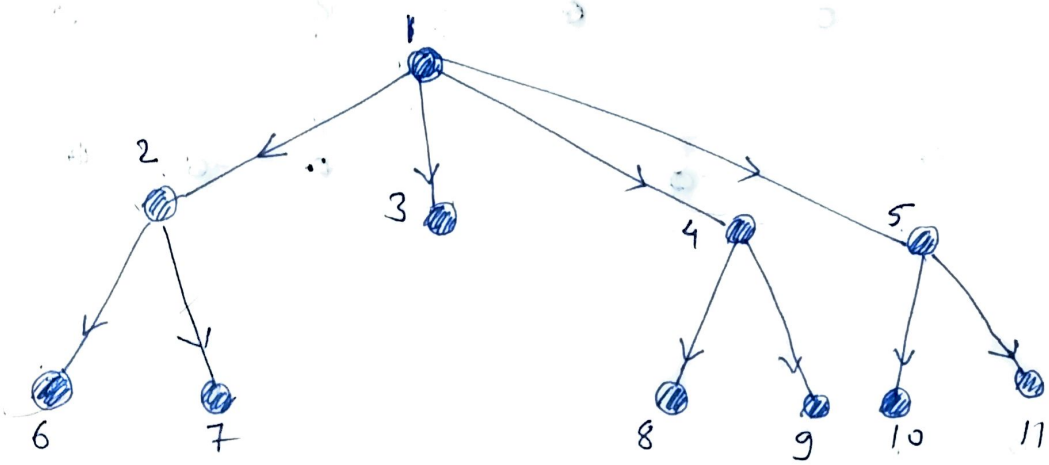
step (II) Draw the edges from node (at level 1) on the right if any. node is situated at same level. and having same mother node

step (III) ~~Draw the~~ Repeate step (II) for the nodes in next level

step (IV) : Rotate the edges connected horizontally clockwise by 45°. Hence we get the req^d binary tree.

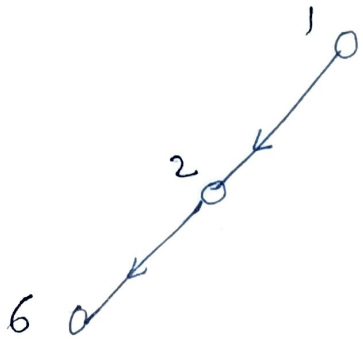
Ex: Draw a directed tree with 4 nodes at level 1 and 6 nodes at level 2. And obtained the corresponding Binary tree

Sol^n:

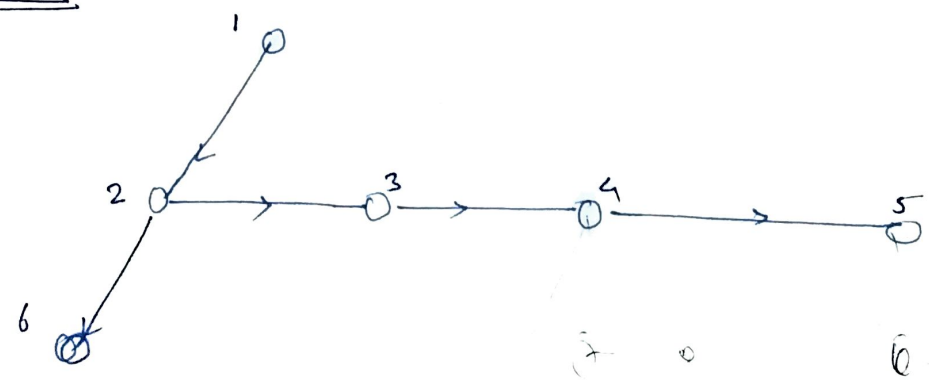


To Draw a Binary tree

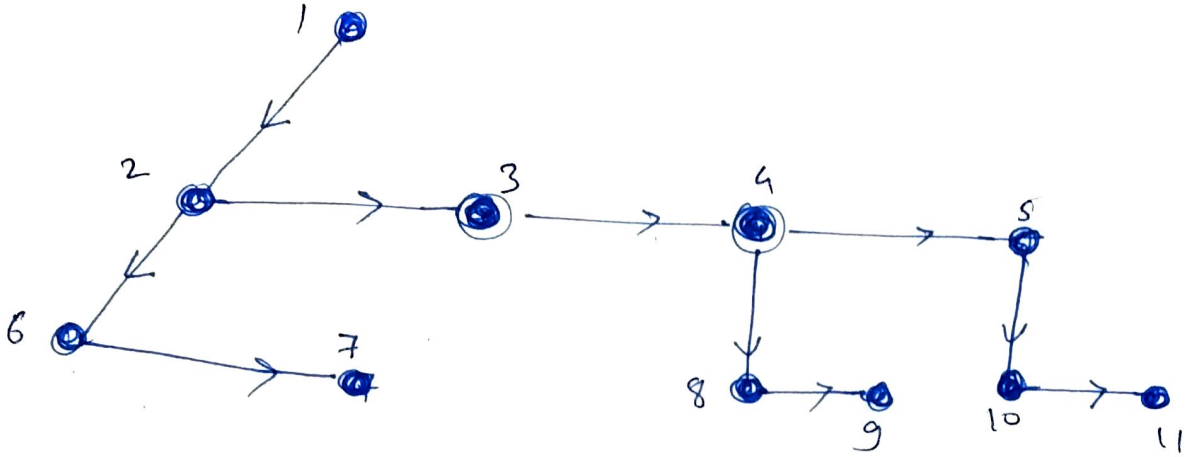
Step (I)



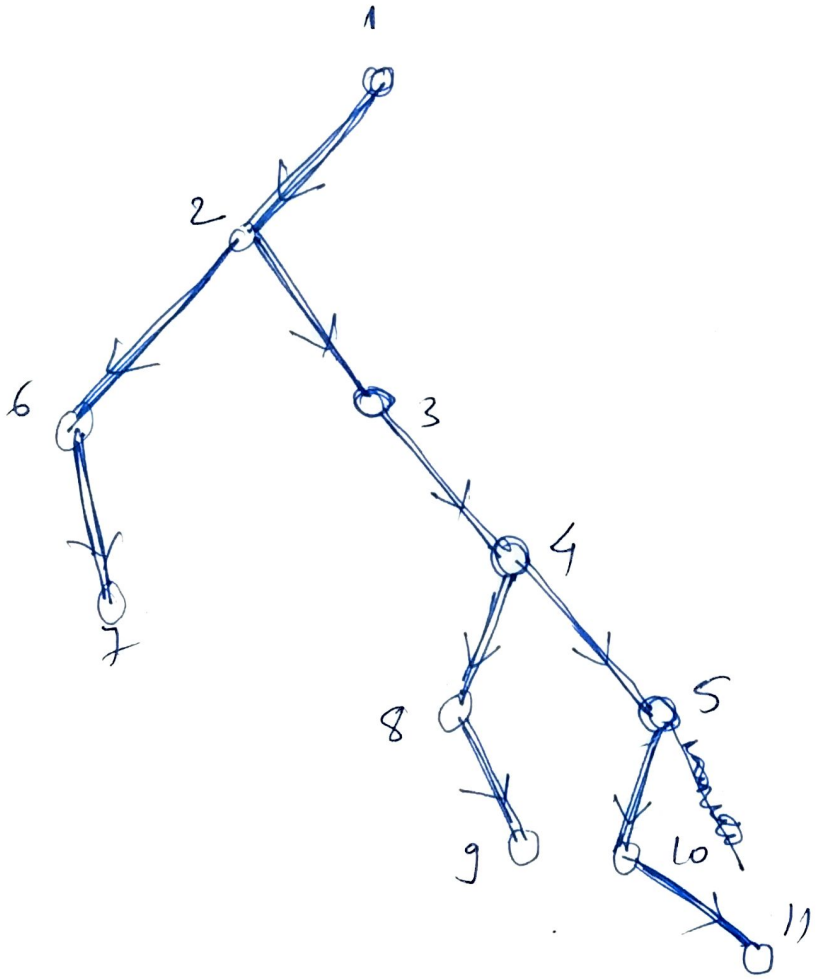
Step (II)



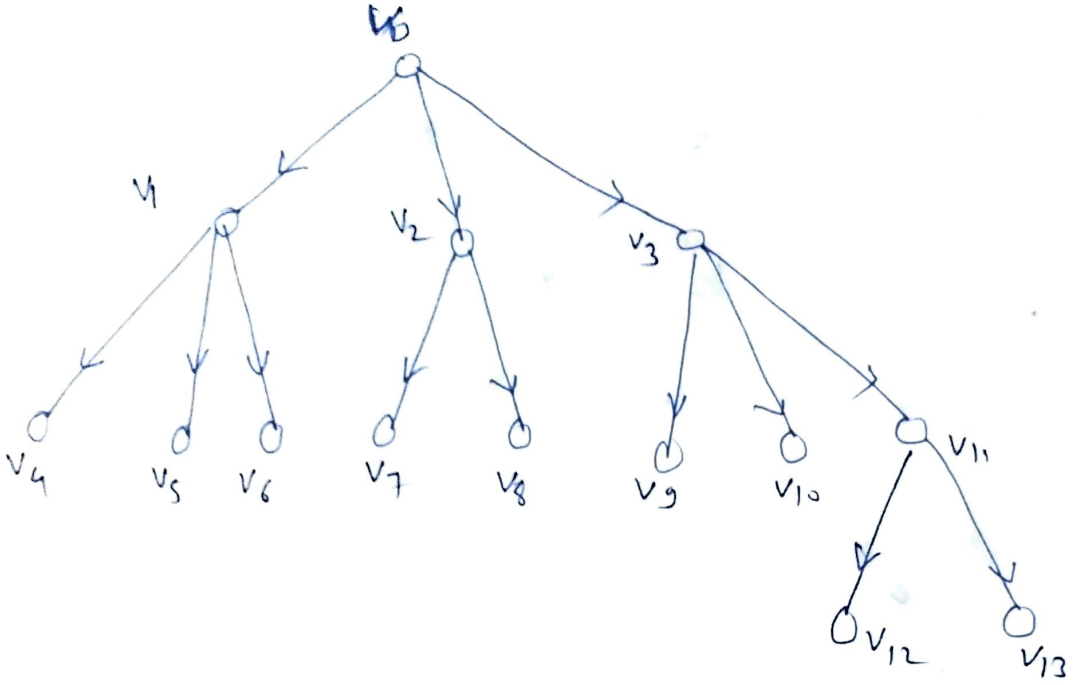
Step (III)



Step (IV)

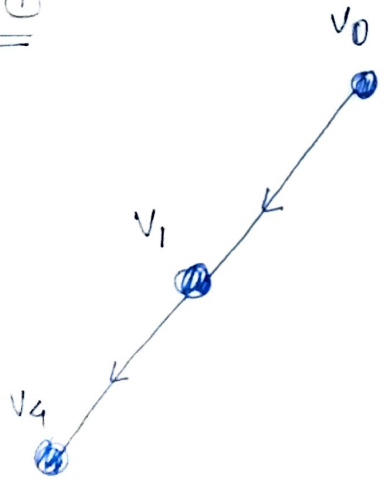


Ex: Draw a Binary tree for the following tree

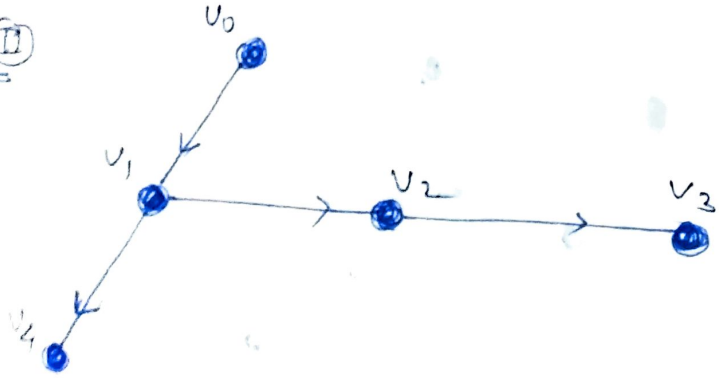


Soln:

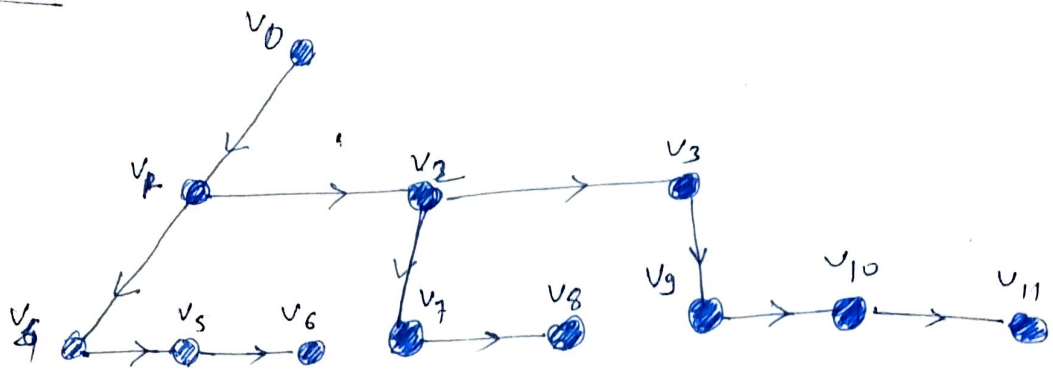
Step (I)



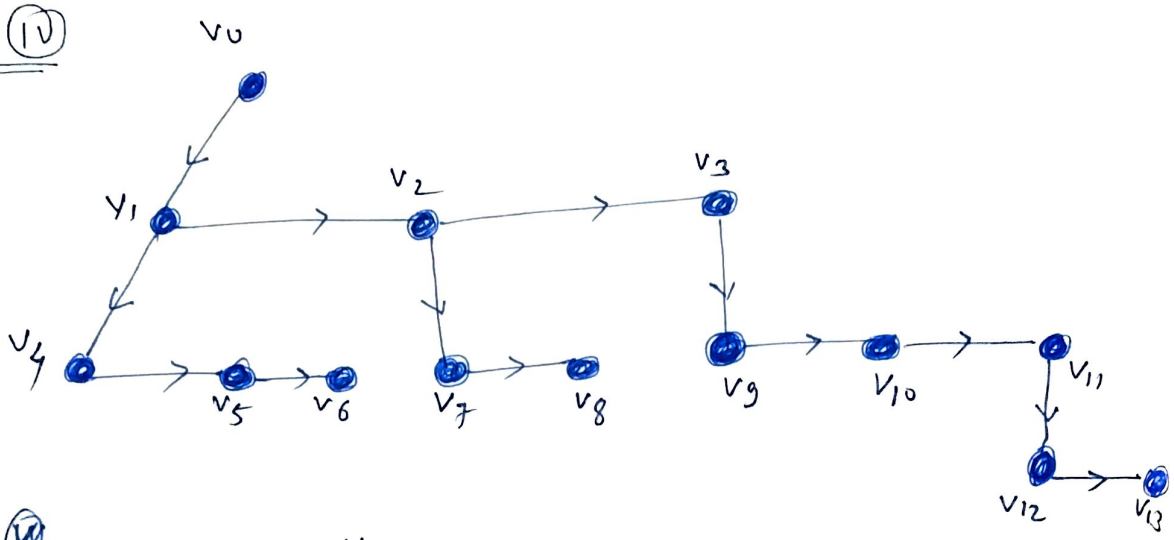
Step (II)



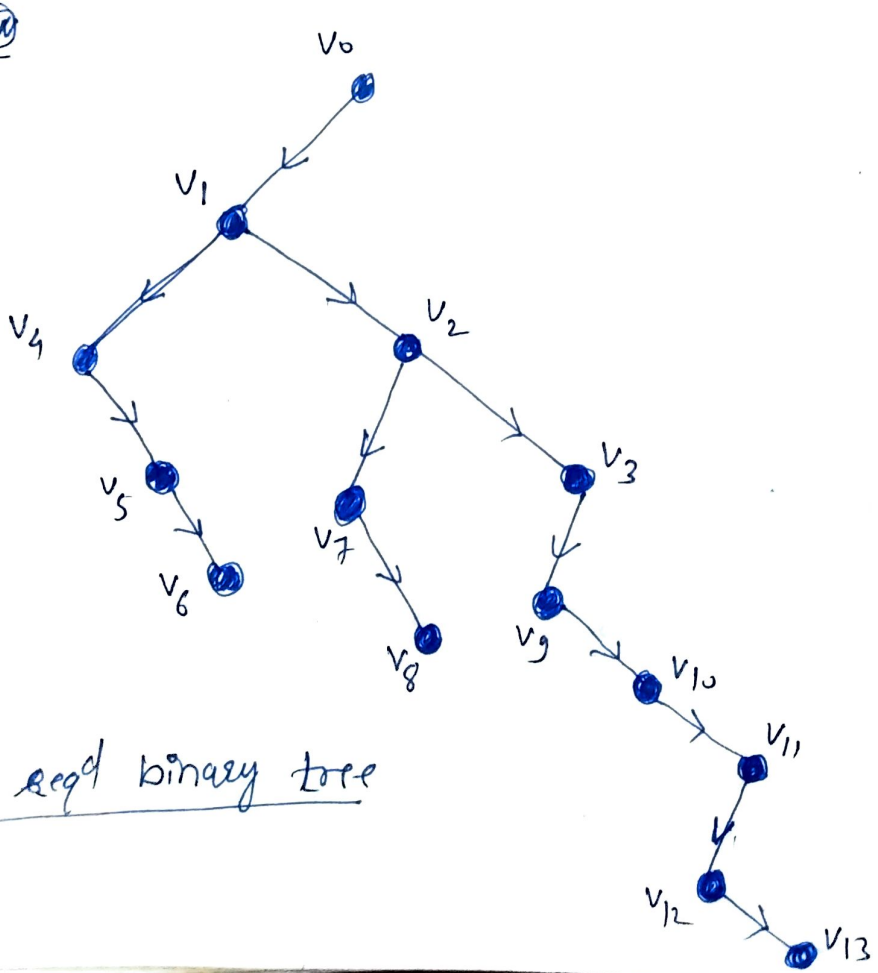
Step (III)



Step (IV)

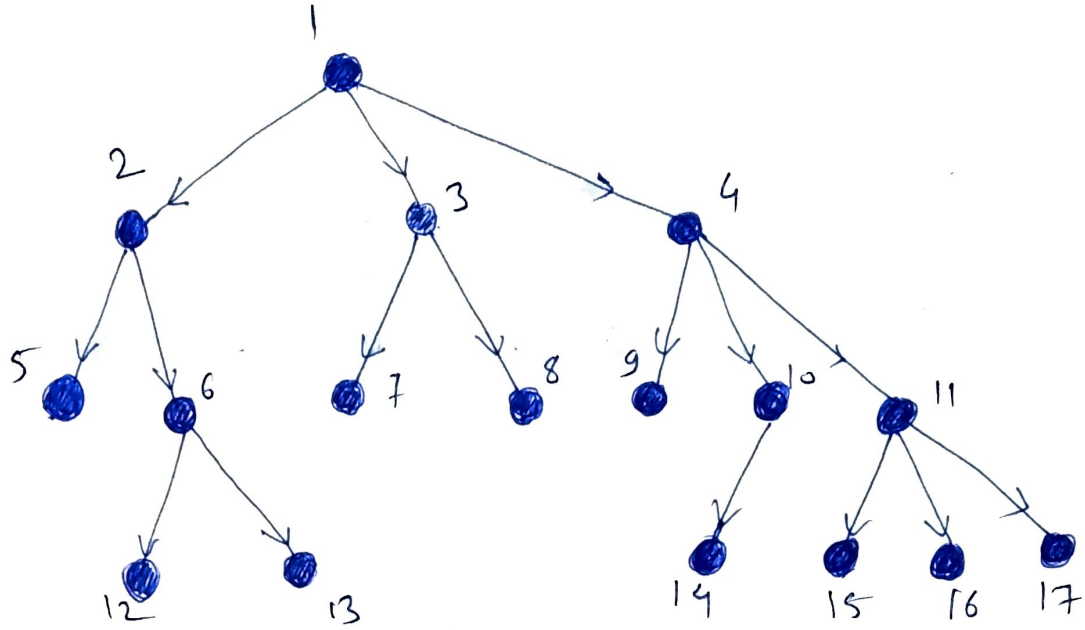


Step (V)



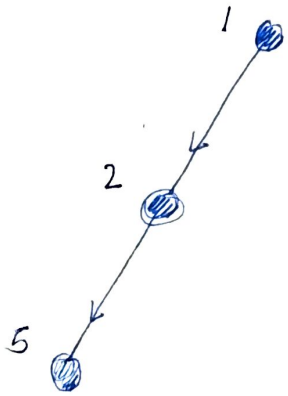
This is reqd binary tree

Ex: Draw a Binary tree for the following tree

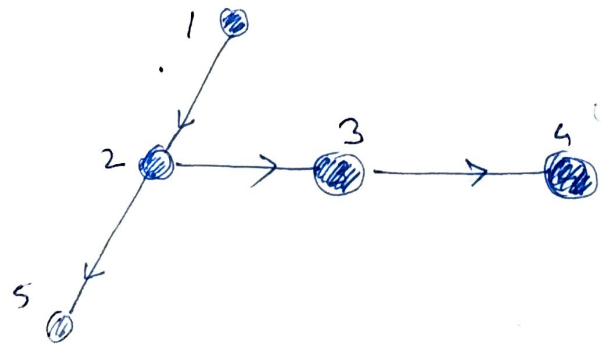


Soln

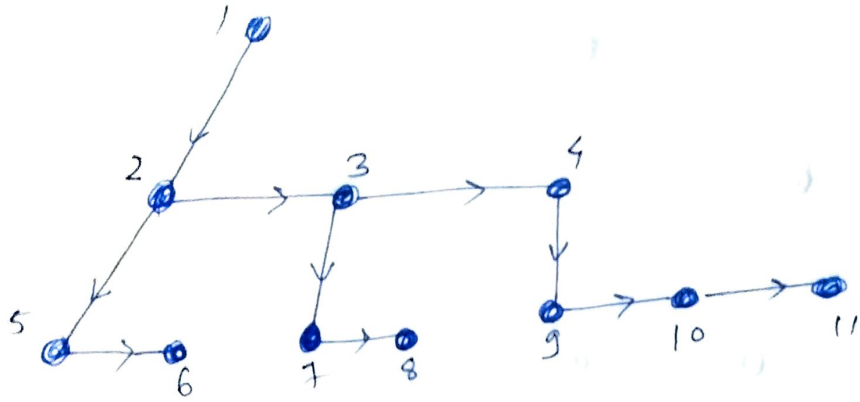
Step (I)



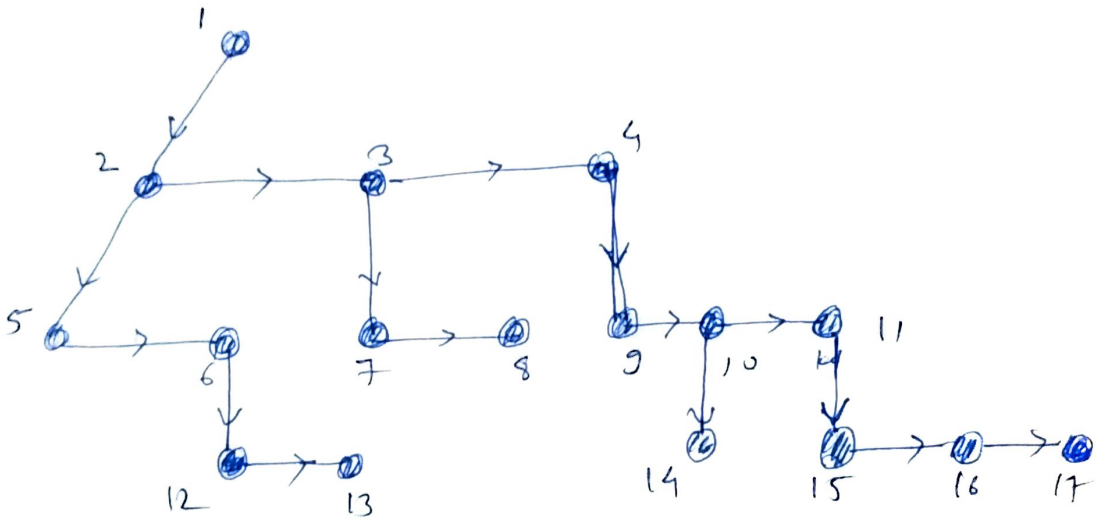
Step (II)



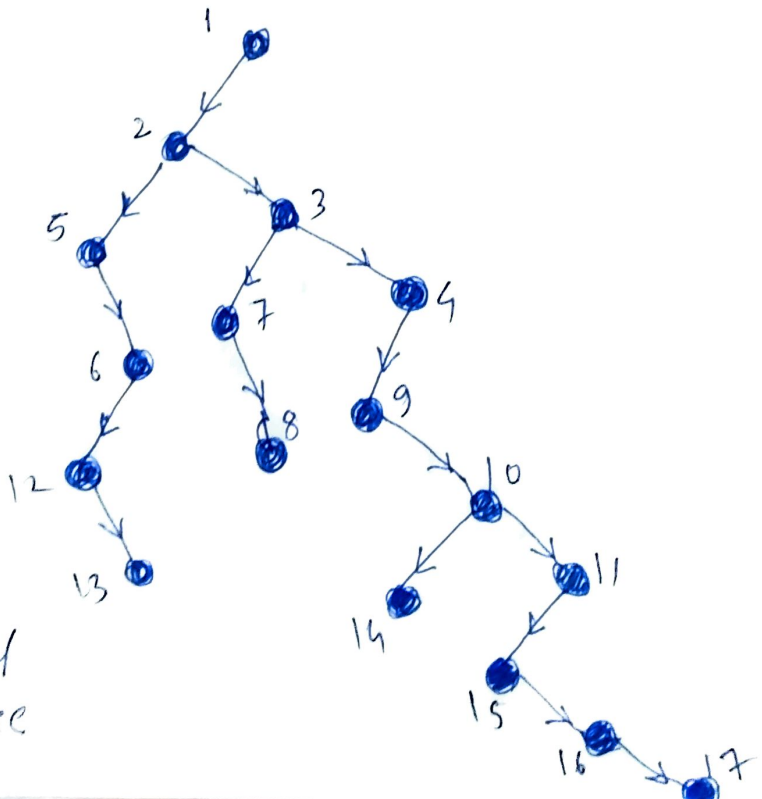
Step (III)



Step (IV)



Step (V)



This is required Binary tree

Ex: Let $R = \{(1,2), (1,3), (1,4), (2,5), (3,6), (3,7)\}$ be the relation defined on set $A = \{1,2,3,4,5,6,7\}$

Find a tree representing a relation R and Hence Find binary tree.

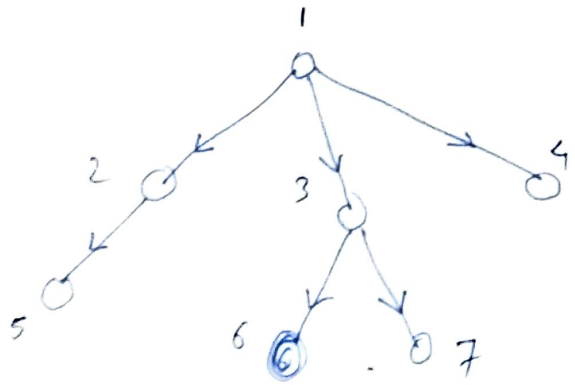
Soln: Given relation set $R = \{(1,2), (1,3), (1,4), (2,5), (3,6), (3,7)\}$

→

Vertex.	outdegree
1	3
2	1
3	2
4	0
5	0
6	0
7	0

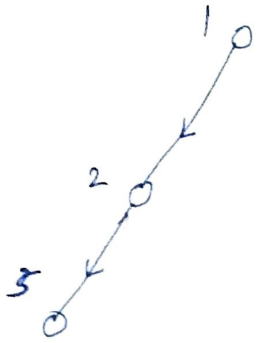
← root is a vertex with maximum outdegree

Req tree is

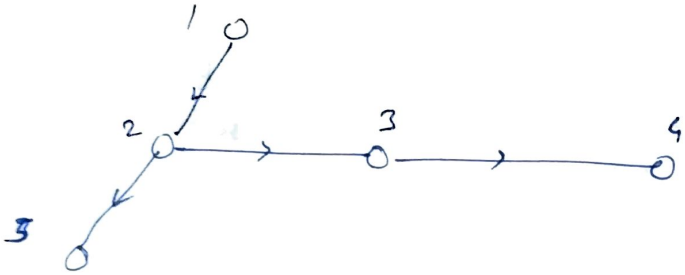


To find binary tree :

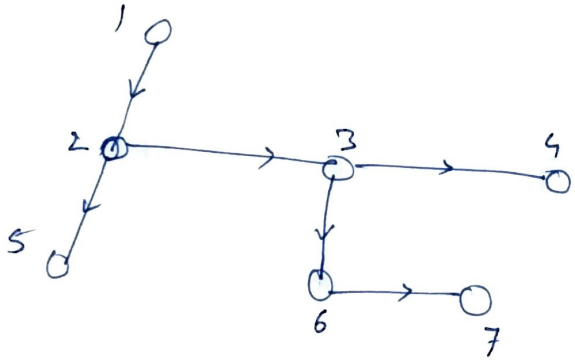
Step (i)



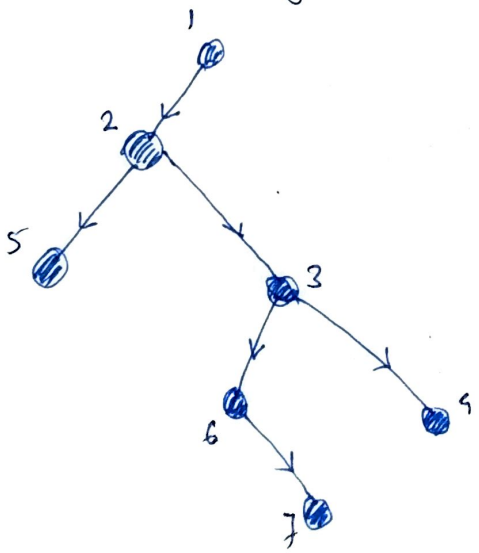
Step (ii):



Step (iii)



Step (iv): Hence req'd Binary tree is



Ex: If $R = \{(a,c), (a,k), (b,j), (d,i), (d,h), (e,a), (e,b), (e,d), (h,g), (h,m), (k,f), (k,l)\}$

be the set of order pair (relation) defined of set

$A = \{a, b, c, d, e, f, g, h, i, j, k, l, m\}$

Find Tree representing a relation R Hence Find their binary tree.

Soln Here: $A = \{a, b, c, d, e, f, g, h, i, j, k, l, m\}$

and relation set defined on A is

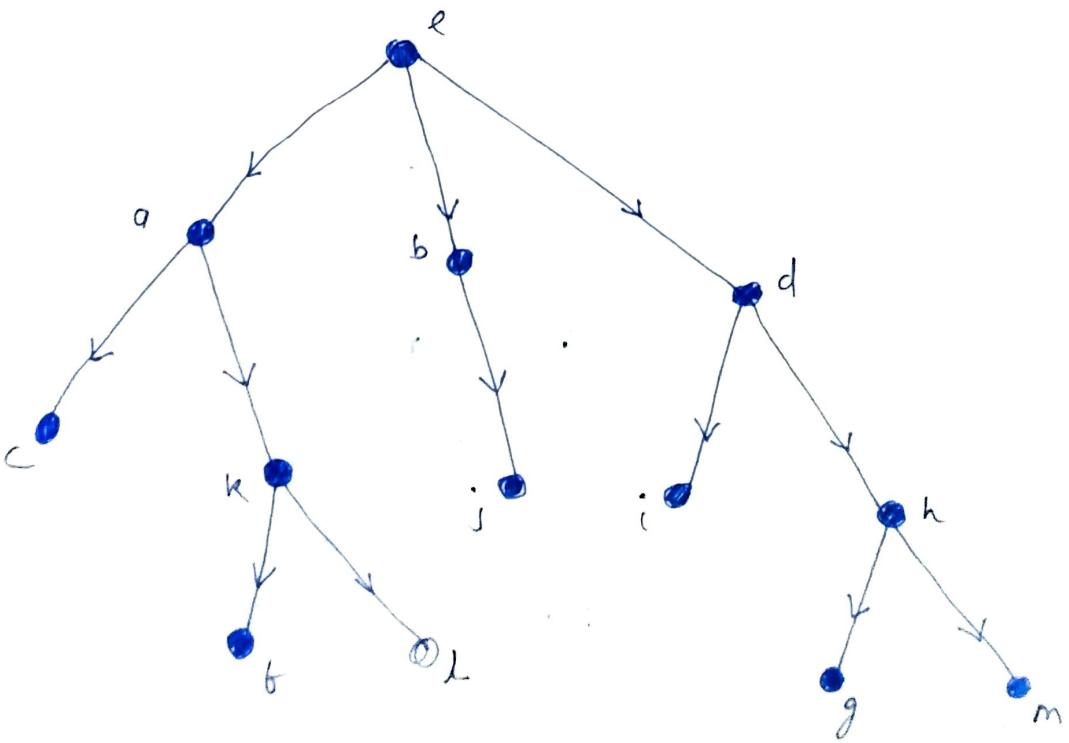
$R = \{(a,c), (a,k), (b,j), (d,i), (d,h), (e,a), (e,b), (e,d), (h,g), (h,m), (k,f), (k,l)\}$

Vertex	a	b	c	d	e	f	g	h	i	j	k	l
outdegree	2	1	0	2	3	0	0	2	0	0	2	0

↑ Root of tree

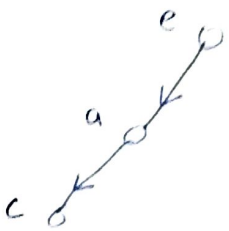
(as it's outdegree is maximum)

Hence Req^d tree is

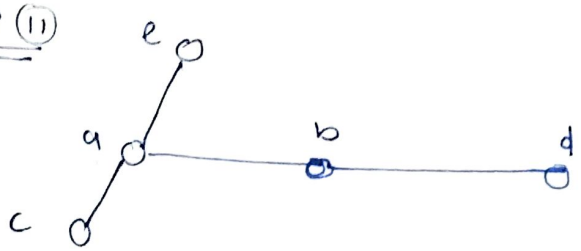


To find Binary Tree :

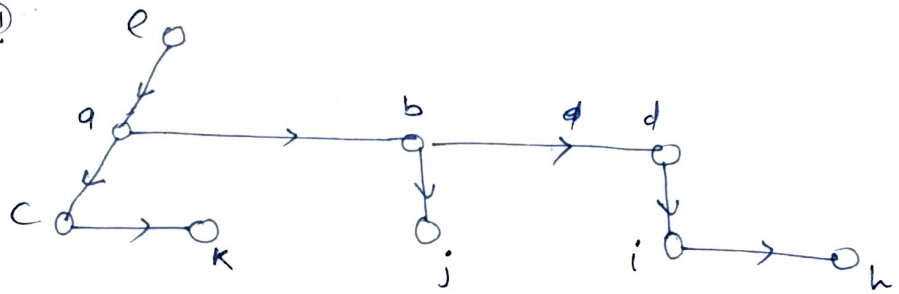
Step (i)



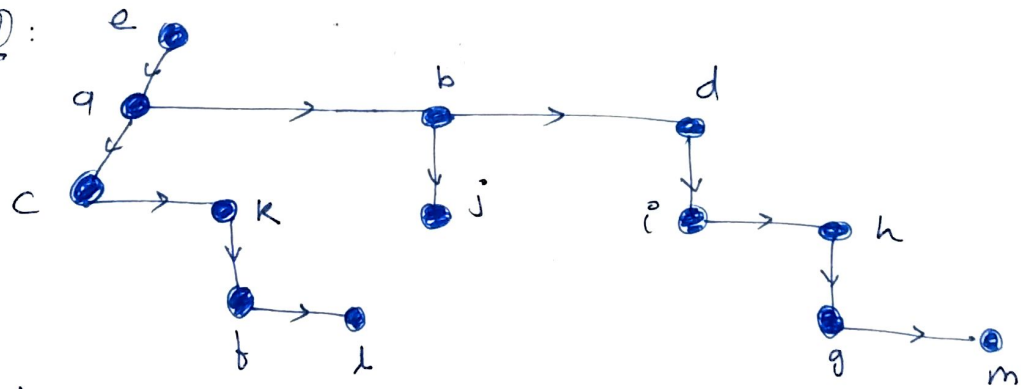
Step (ii)



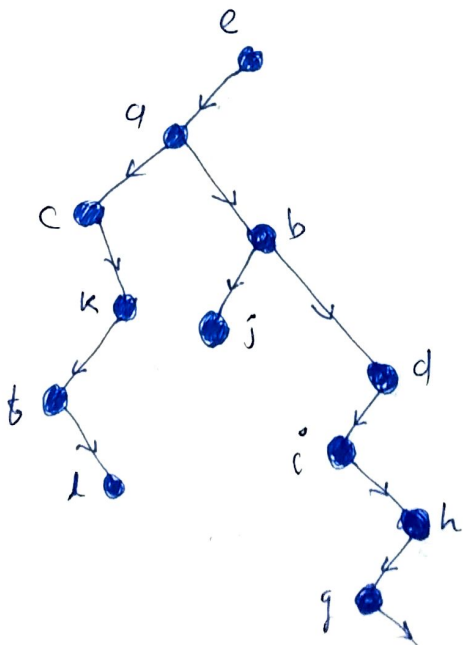
Step (iii)



Step (iv) :



∴ Req^{d} binary Tree is



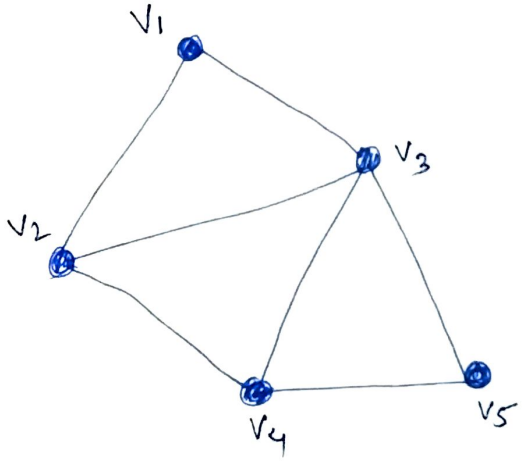
Subgraph :

A graph obtained by deleting some vertex or some edges from main graph $G(V, E)$ is called as subgraph of G and it is denoted by $H(V', E')$

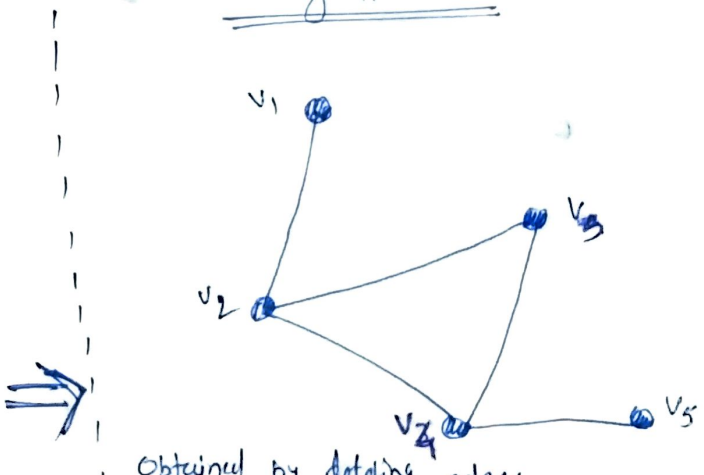
i.e. If $V' \subseteq V$ & $E' \subseteq E$ then a graph $H(V', E')$ is said to be subgraph of graph $G(V, E)$.

for eg :

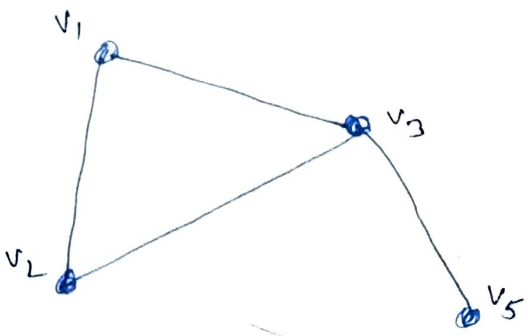
Graph G



Sub graph H



Obtained by deleting edges (v_1, v_3) & (v_3, v_5)



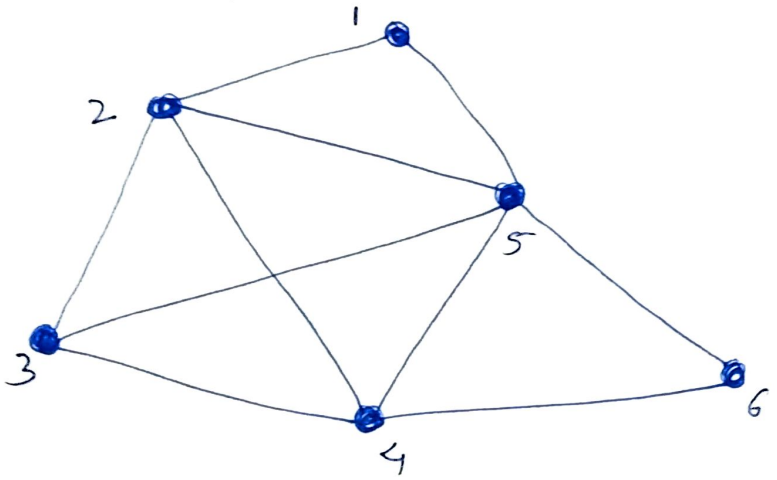
Obtained by deleting vertex v_4

Spanning of graph

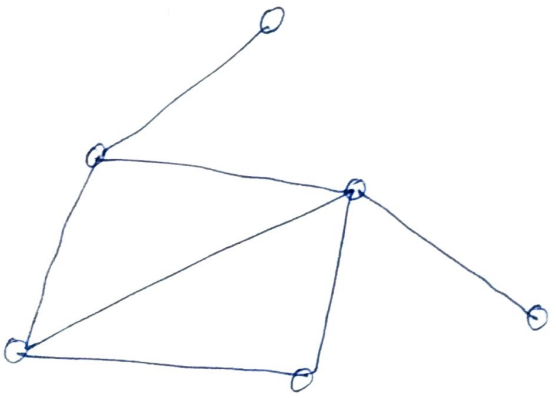
It is a subgraph obtained by deleting some edges. i.e. If $G(V, E)$ be the graph and $H(V', E')$ be the subgraph. Then H is said to be spanning of G if no. of vertices $V' = V$ and $E' < E$

Eg:

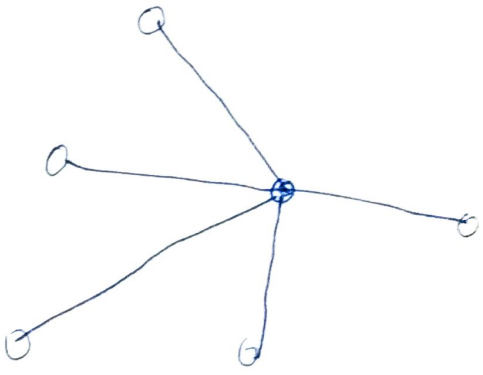
If Graph



Then



(1)



(2)

etc. are spanning of graph G.

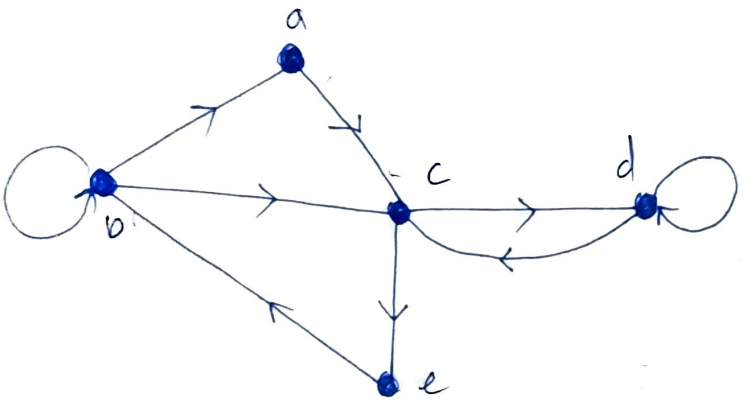
Spanning Tree :⇒

A spanning tree is a spanning graph in which there is no any cycle

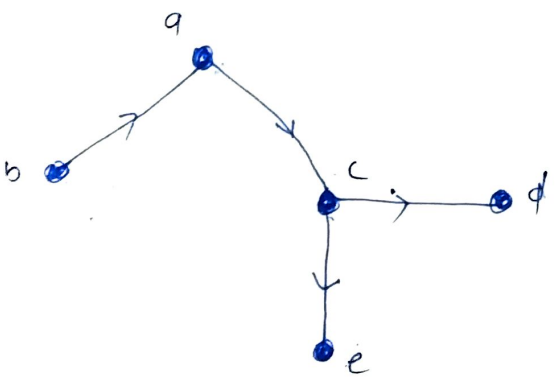
OR

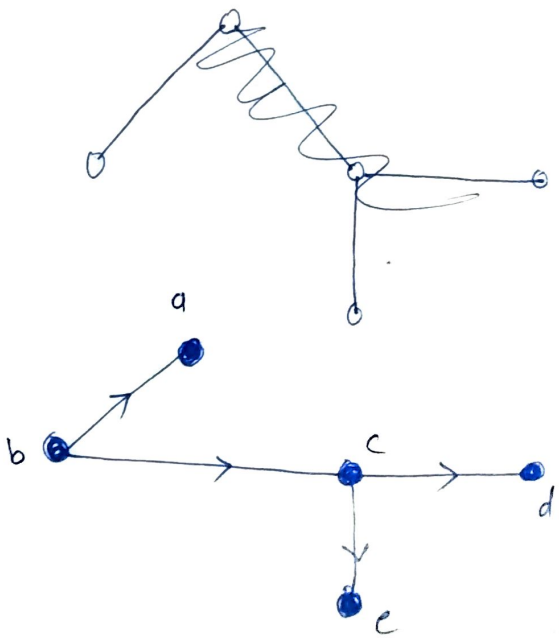
It is a acyclic spanning graph. There may be more or one spanning tree for a given graph.

for eg.



∴ spanning trees are





Minimal Spanning Tree

A spanning tree of a connected weighted graph (Tree) such that the total weight of edges in the tree is minimum. is called as minimal spanning tree.

There are two algorithm to find minimal spanning tree.

(i) Kruskal algorithm :

(ii) ~~Prims~~ Prim's algorithm

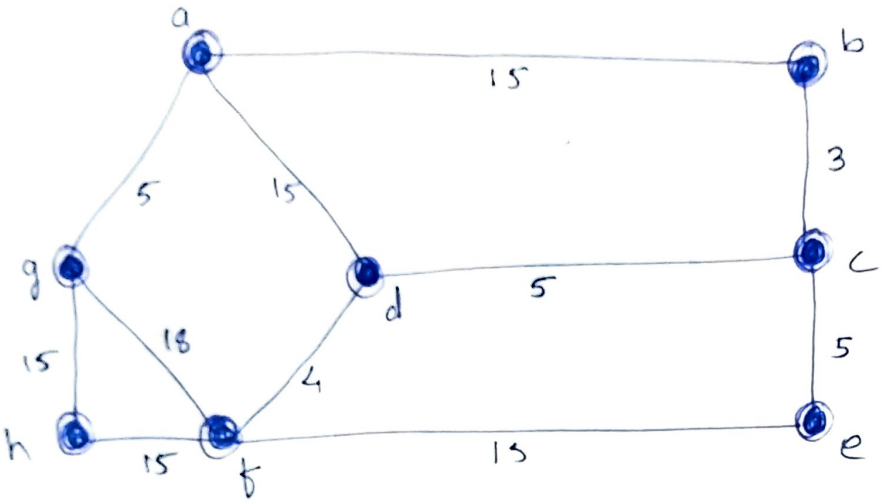
Kruskal algorithm

Step (i): Arrange all edges of G in an increasing order of their weight. choose ~~among them~~ an edge e_i with ~~least~~ minimal weight. and add this edge in Tree 'T'

Step (ii): choose another edge e_j with next ~~highest~~ minimal weight in such a way that these edges do not form a cycle. If e_j does not form a cycle, then add e_j also in tree 'T'. otherwise exclude this edge.

Step (iii) Repeat step (2) until all $(n-1)$ edges are added in T. and do not form a cycle.

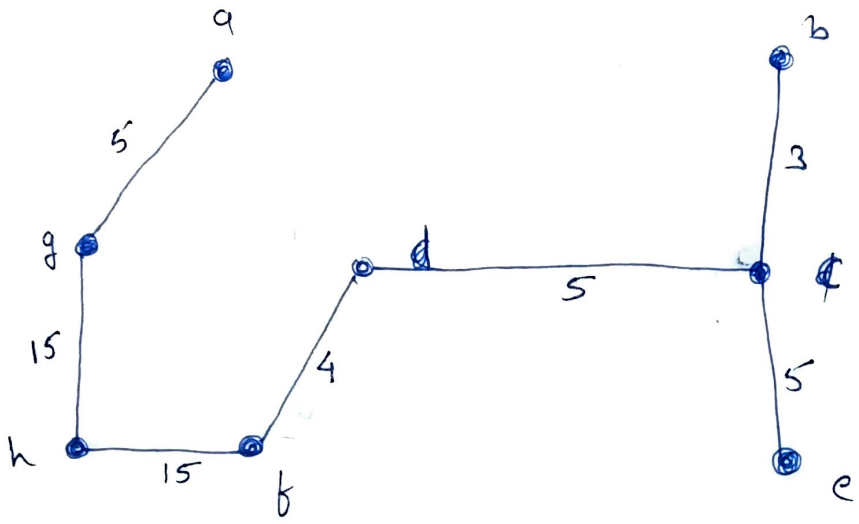
Ex: Determine a railway network of minimum cost for the cities in the following figure.
Indicate the minimal cost.



Soln:

edges	weight	
(b c)	3	← add
(d, b)	4	← add
(a, g)	5	← add
(c, e)	5	← add
(d, c)	5	← add
(g h)	15	← add
(h, f)	15	← add
(f e)	15	← not added [exclude]
(a, d)	15	← not added [Exclude]
(a b)	15	← not added [Exclude]
(g f)	18	← not added [Exclude]

Reqd minimum-spanning tree

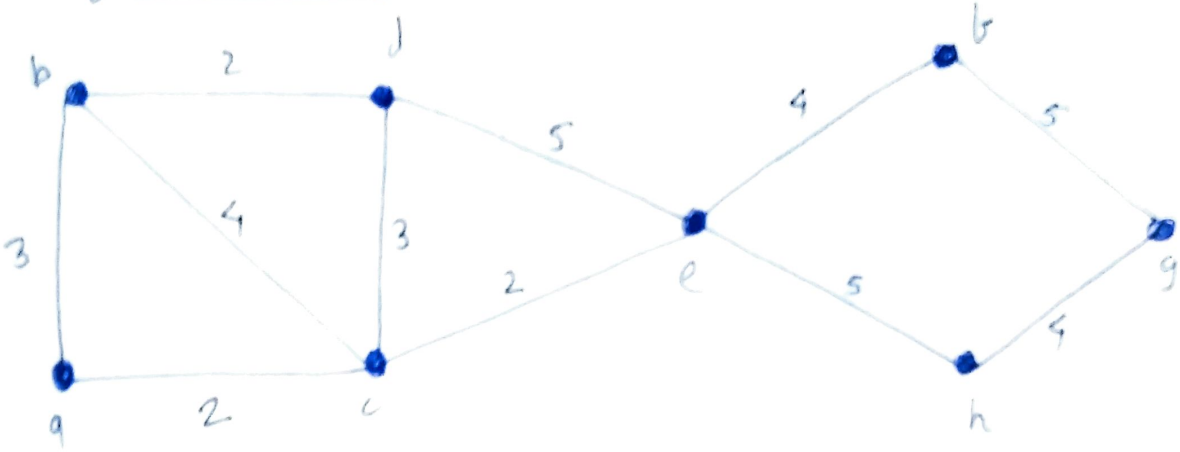


Hence minimal weight of tree is

$$= 5 + 15 + 15 + 4 + 5 + 3 + 5$$

$$= 52$$

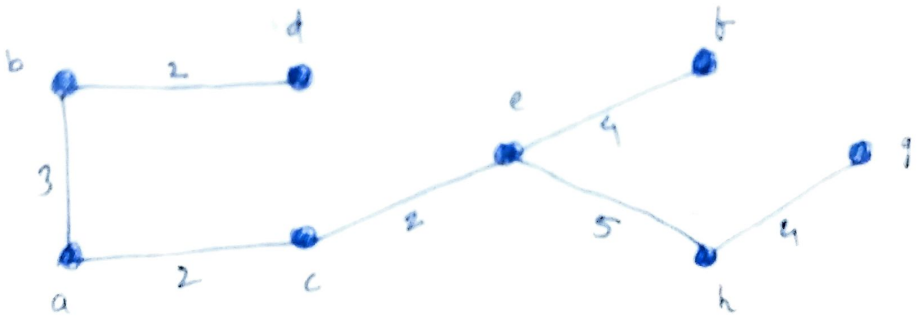
Ex: Find the minimal spanning tree for the following weighted graph hence find minimum weight using Kruskal-algorithm



Sol:

Edges	Weight	
(b, d)	2	← add
(a, c)	2	← add
(c, e)	2	← add
(b, a)	3	← add
(d, e)	3	← Not add
(b, e)	4	← not add
(e, f)	4	← add
(h, g)	4	← add
(d, e)	5	← not add
(e, h)	5	← add
(f, g)	5	← not add

Req: minimal spanning tree

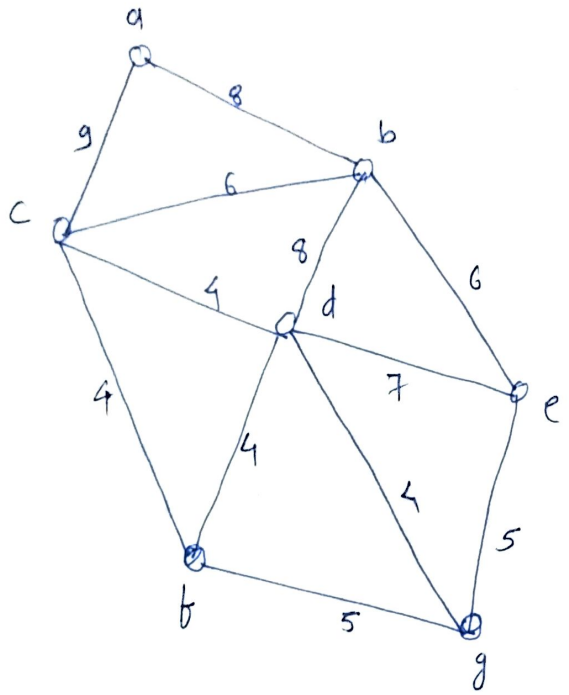


∴ minimum weight = 2 + 3 + 2 + 2 + 4 + 5 + 4

Hw:

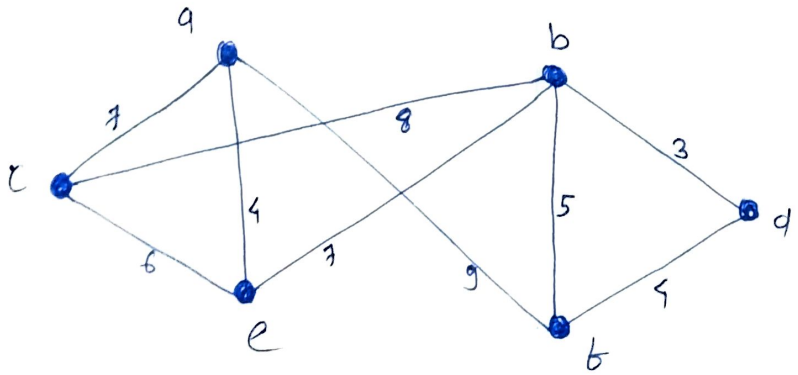
① Find the minimal spanning tree for the following weighted graph also find their minimum weight by using Kruskal algorithm

(i)



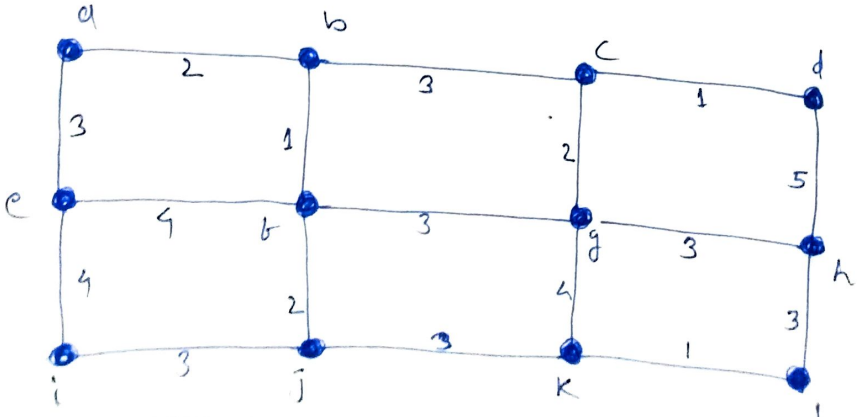
Ans

(ii)



Ans
24

(iii)



23

☆ Prim's Algorithm ☆

Step (I): Select an edge in G with minimum weight and introduce it in minimal spanning tree 'T'.

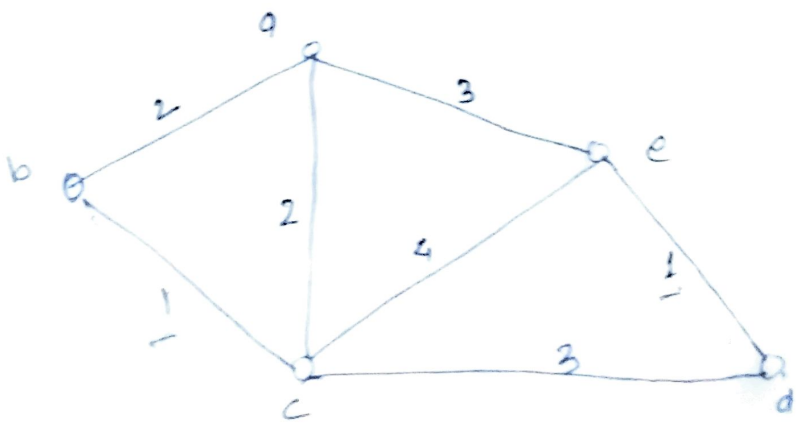
Step (II): Find all the edges incedence with the vertices of selected edge. and then select an edge with minimum weight and include it in 'T'

Step (III): Repeate the step (II) to add edges with minimum weight in Tree 'T'. provided it not formed any cycle.

Step (IV): Repeat this process until all the vertices of Graph G are included in T. and total number of edges equal to (n-1). If n is total no. of vertices

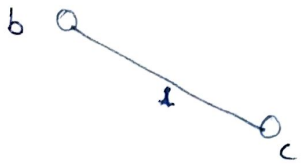
---

Ex: Find a minimal spanning tree for the following graph using prims algorithm.



Soln:

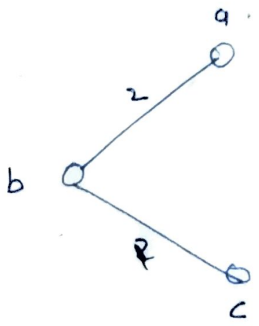
Step (I): Select an edge (b c) having weight 1



Step (II): Edges incedence with Vertex b and c are as follows

(b a)	2	← add
(c a)	2	← not add
(c, e)	4	
(c, d)	3	←

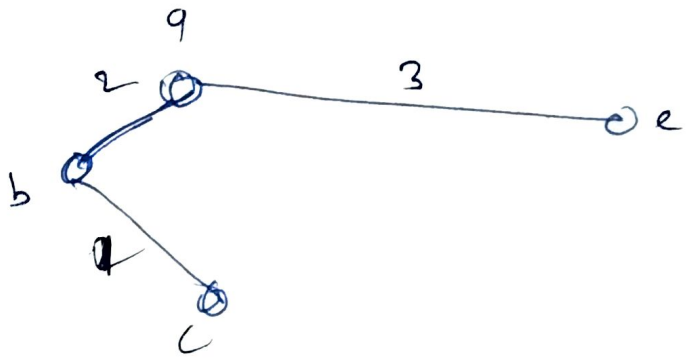
add edge (b, a) with weight 2 in the tree T. only



Step (III): Now consider an edges incedence with vertex a, b, & c as follows

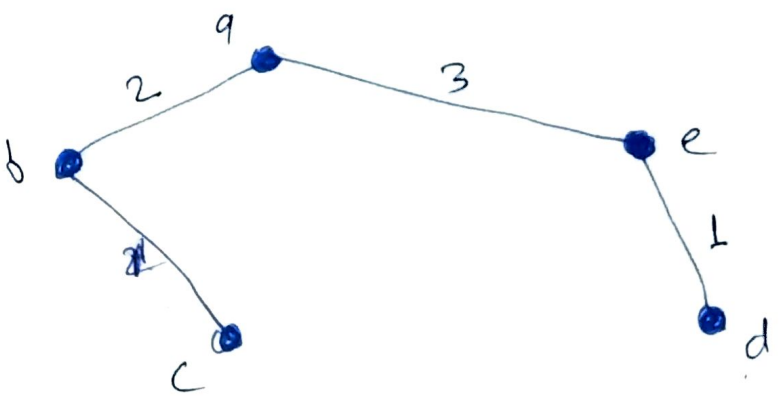
(a, c)	2	← not add (as it form a cycle)
(a, e)	3	← add
(c, d)	3	←
(c, e)	4	← (as it form a circle)

add edge (a, e), ( )



Step (IV) Now again consider an edges incedence with the vertex a, b, c, & e as follows (which is not add in T)

~~(a e)~~	~~2~~	Excluded
(c, e)	4	
(c, d)	3	← add (∵ minimum weight)
(e, d)	1	

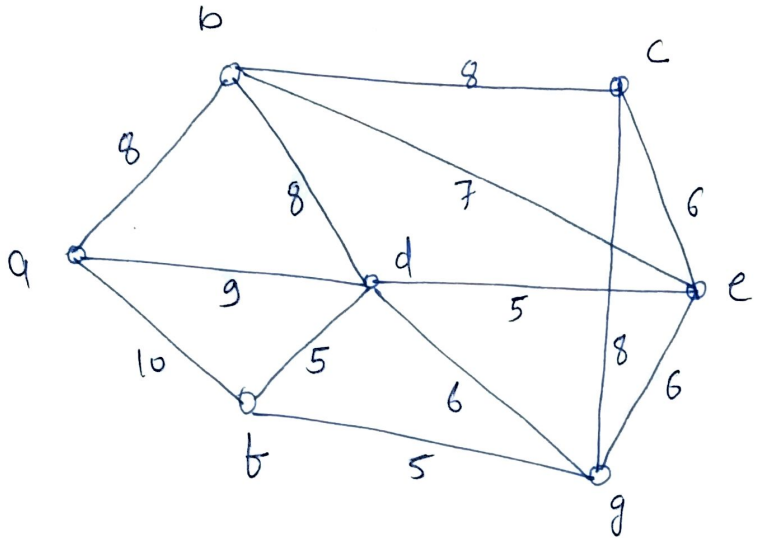


This is required minimal spanning tree

and minimum weight = 2 + 4 + 3 + 1

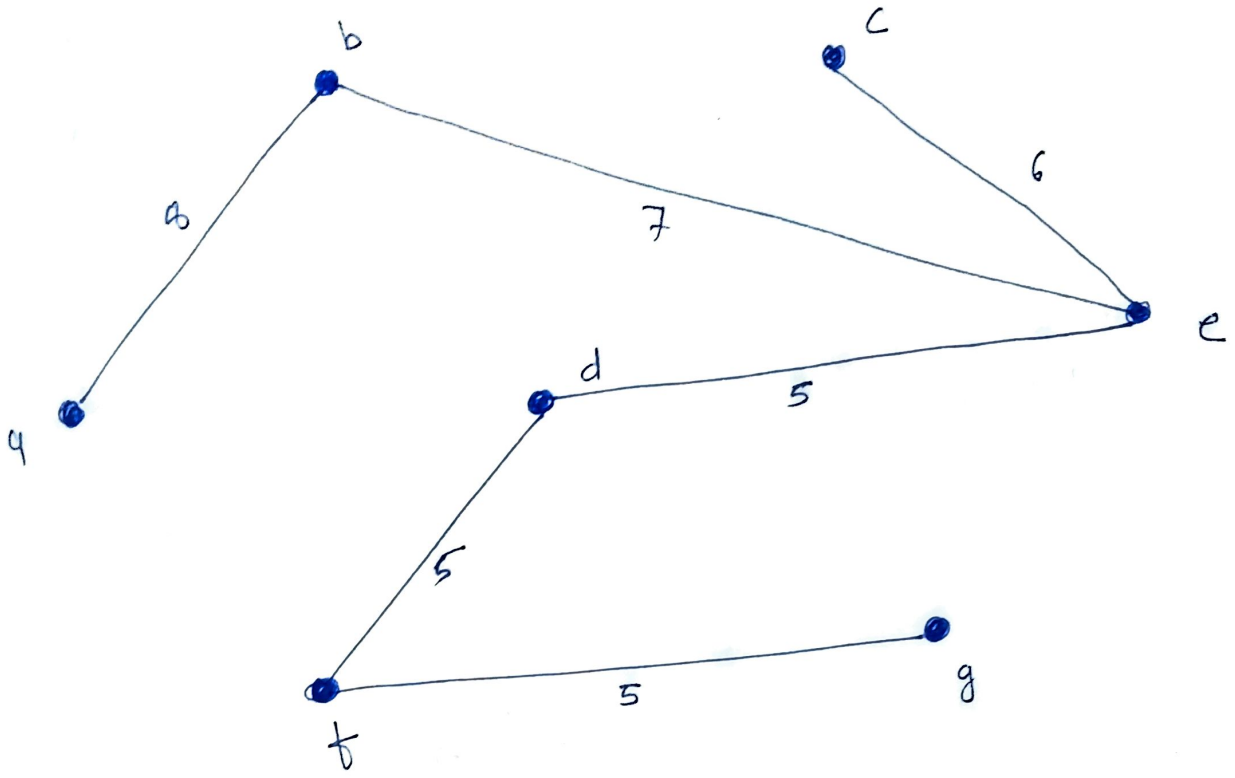
= 7

Ex: Obtained minimal spanning tree for the following weighted graph also find the minimum weight using prim's algorithm.



Soln: using prim's algorithm, we add following edges in the tree as follows

edges	weight
(d, f)	5
(f, g)	5
(d, e)	5
(e, c)	6
(e, b)	7
(a, b)	8
minimum weight	36



minimum weight = 36